Debugging Strategies

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Transparent Book Recommender

THE CONNECTED WEEK

Choose a revealing test. Repair one cause. Rerun the suite.

Test returned values against independent expectations, expose coincidental passes and preserve failures during controlled repair.

Keep one habit: Fix the rule and expectations → compare actual values → repair and retest without hiding failures.

MAIN LESSON · SECTION 1

Which test would distinguish the suspected bug?

> Use independent expectations and targeted changes rather than patching the answer.

Start with a coincidental pass

A wrong function can pass a test by coincidence. Give every test a purpose, calculate expectations from the rule and keep the observed result even when it fails.

Recall contracts and subtraction

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

For capacity 9 and used 2, what should remaining return?

Your learning goals

- I can design revealing tests for a function.
- I can compare expected and actual results.
- I can repair a defect and rerun earlier cases.

▼ Earlier wording, alternatives and picture transcripts

Your learning target

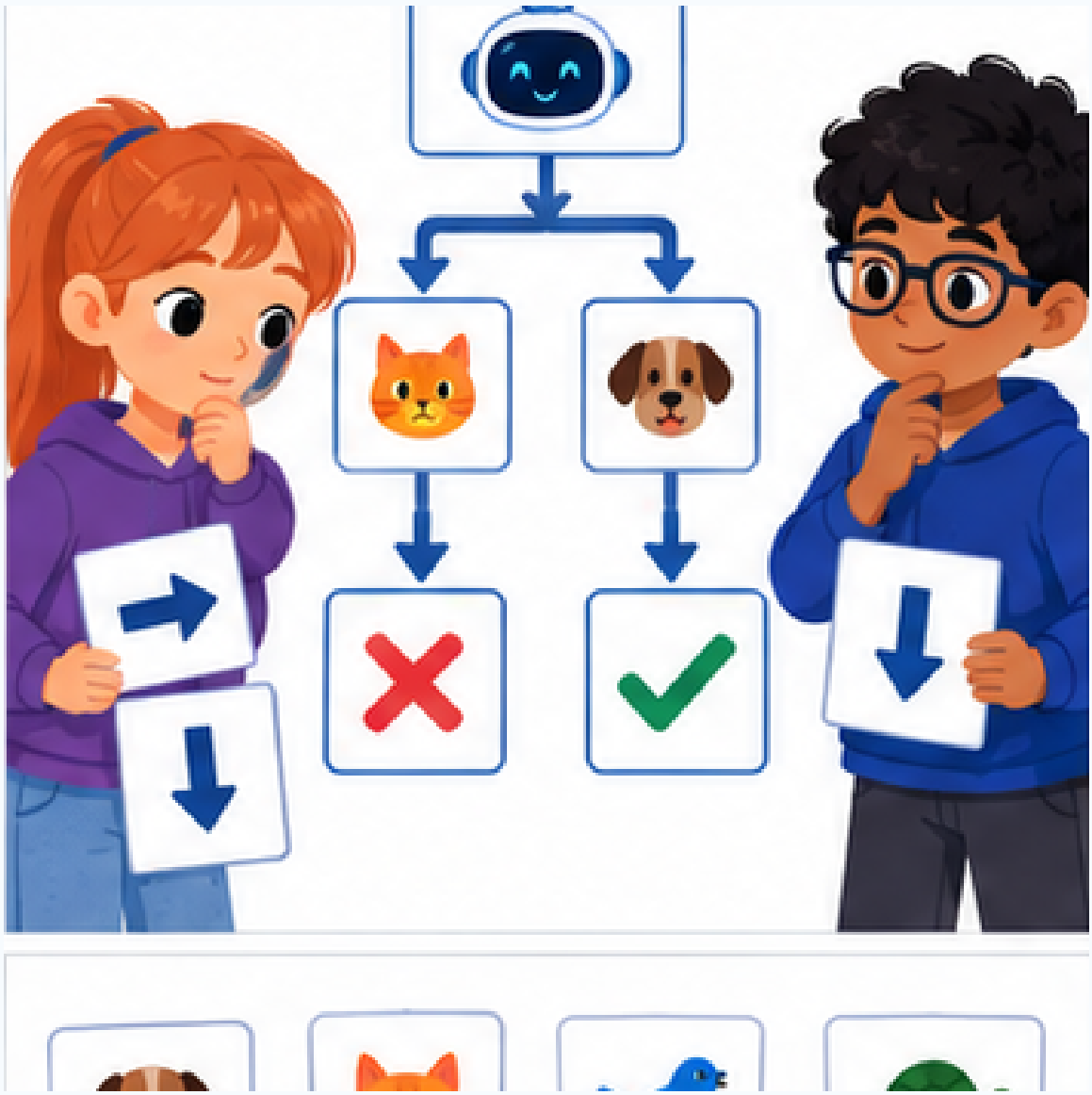
Remember before reading

Without opening the old lesson, explain one key idea from Loops and Stopping. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Branching Logic. Give an example and a mistake that the idea helps you avoid.

Week 6: **Debug**ging Strategies

Good **debug**ging isolates a cause with **controlled test**s.



THINKER MISSION

Use divide-and-check to locate a faulty branch.

Thinker name and date:

> **Write or draw your reasoning and evidence.**

Readiness for the connected explanation

This week’s end goal

By the end, I can test a function with expected values that distinguish a particular bug.

MAIN LESSON · SECTION 2

State the remaining-space contract before testing

> The main function subtracts used spaces from capacity.

The written rule and allowed relationship

A function can pass one example by coincidence. Start with a **written rule**, calculate expected results independently, and include ordinary, zero and **boundary** cases.

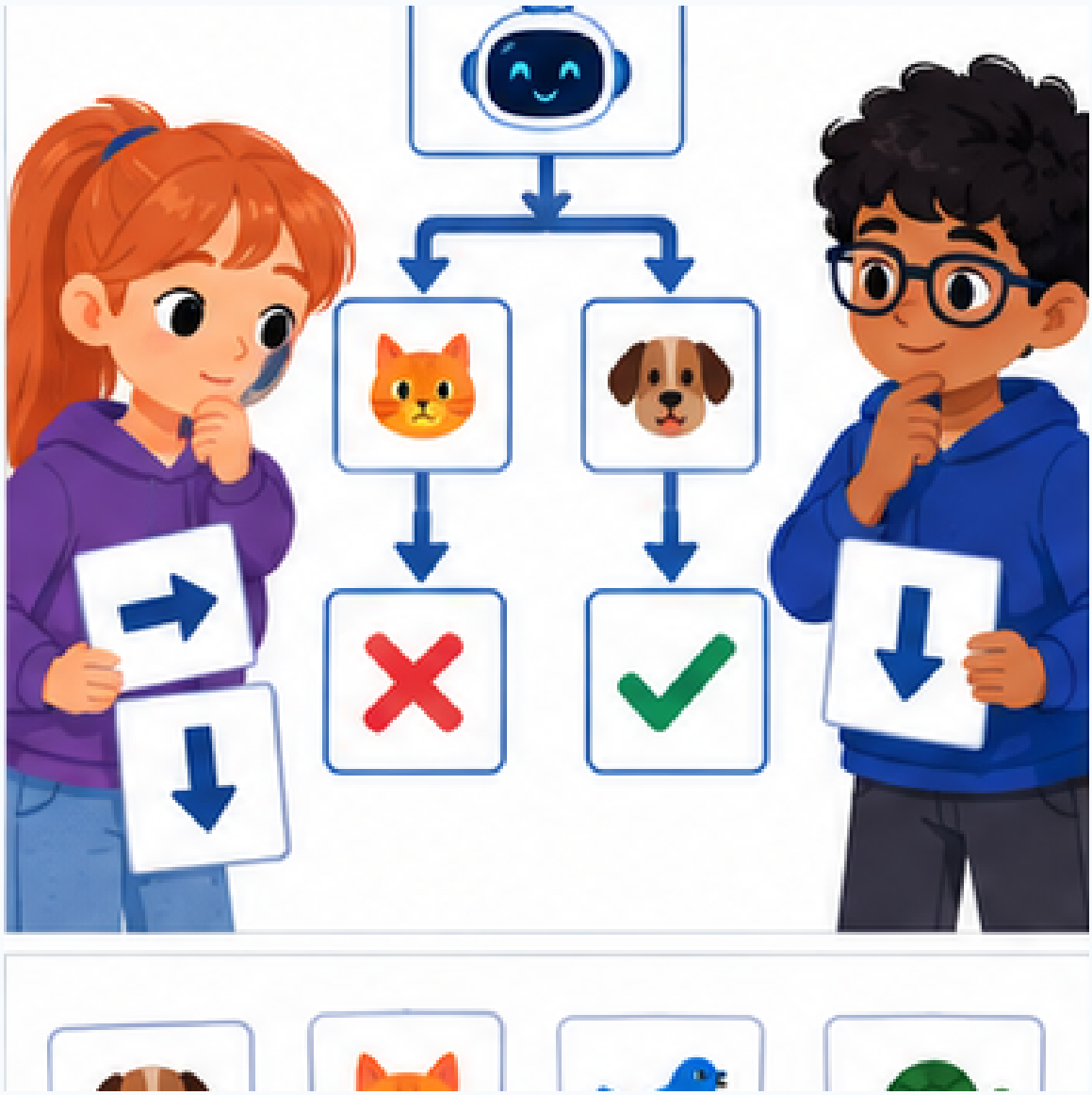
We will test a subtraction function under the contract 0 <= used <= capacity.

▼ **Earlier wording, alternatives and picture transcripts**

PICTURE STORY

Notice → Predict → Explain

Begin with visible **evidence**. Do not explain more than the picture and information support.



MYSTERY

The learners must use **debug**ging strategies to solve a problem. What information do they have? What decision or calculation do they make? What would count as **evidence** that it worked?

Three **observation**s (what you can point to):

Write or draw your reasoning and evidence.

My **prediction** and the clue supporting it:

Write or draw your reasoning and evidence.

A closer explanation

MAIN LESSON · SECTION 3

Run the complete function test harness

> Read returned values, tuple unpacking and equality checks separately.

The complete executable function and tests

```
def remaining(capacity, used):
    return capacity - used

cases = [(8, 3, 5), (8, 8, 0), (8, 0, 8)]
for capacity, used, expected in cases:
    actual = remaining(capacity, used)
    print(actual == expected)
```

Give each input/expected tuple a purpose

Follow the evidence

STEP OR CASE	EVIDENCE	MEANING
Ordinary case	8 and 3	Expect 5
Full case	8 and 8	Expect 0
Unused case	8 and 0	Expect 8

Read each row from left to right and connect the evidence to the stated rule.

A useful test includes an **input** and an answer calculated from the **written rule** before the program runs. For 8 spaces with 3 used, expect 5 left. For 8 used, expect 0. For none used, expect 8.

These cases ask different questions. An incorrect addition rule happens to pass the unused case because adding zero and subtracting zero give the same result. That is why one passing example is weak **evidence**.

Explain True results and first-failure assertions

- Each parenthesized group is a tuple, an ordered group of values. The loop unpacks each group into three names. Equality compares the actual result to the independently calculated expected result.

  The worked script prints True three times. An assert actual == expected could instead stop at the first mismatch with **Assertion**Error.

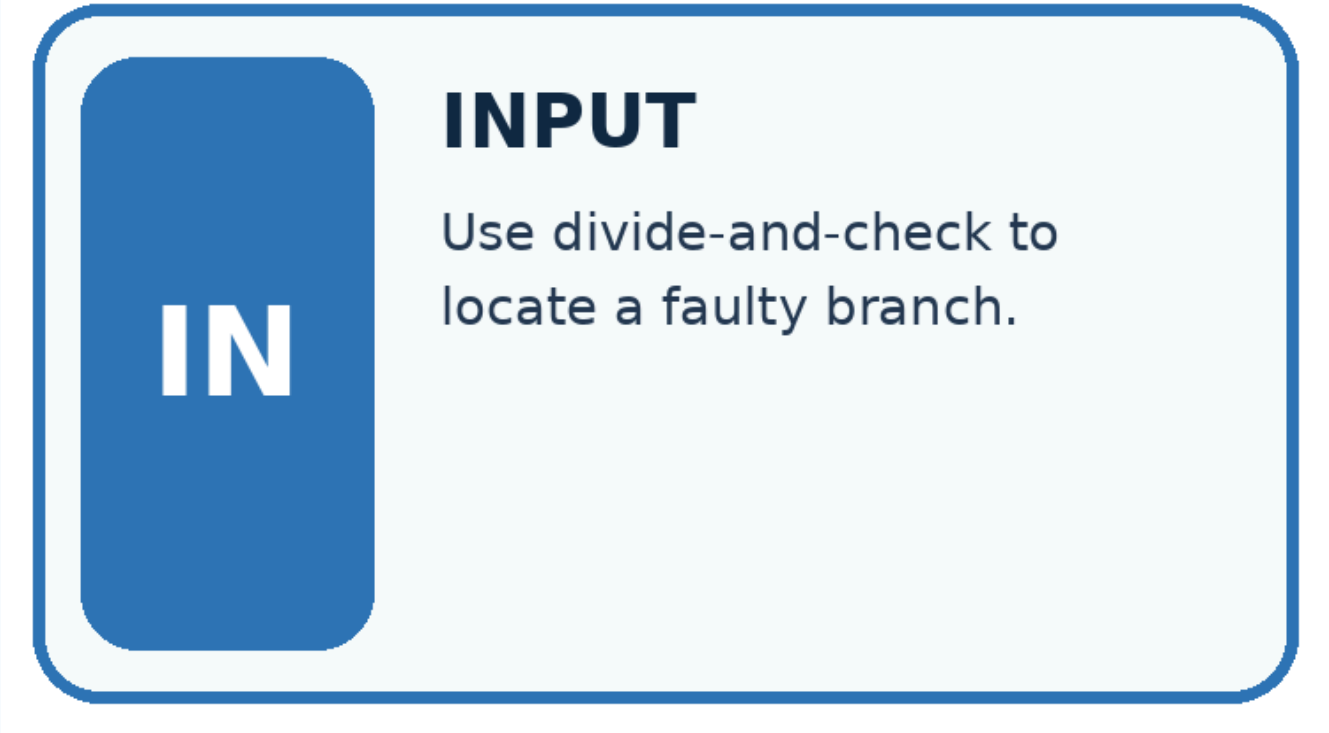
- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ **Earlier wording, alternatives and picture transcripts**

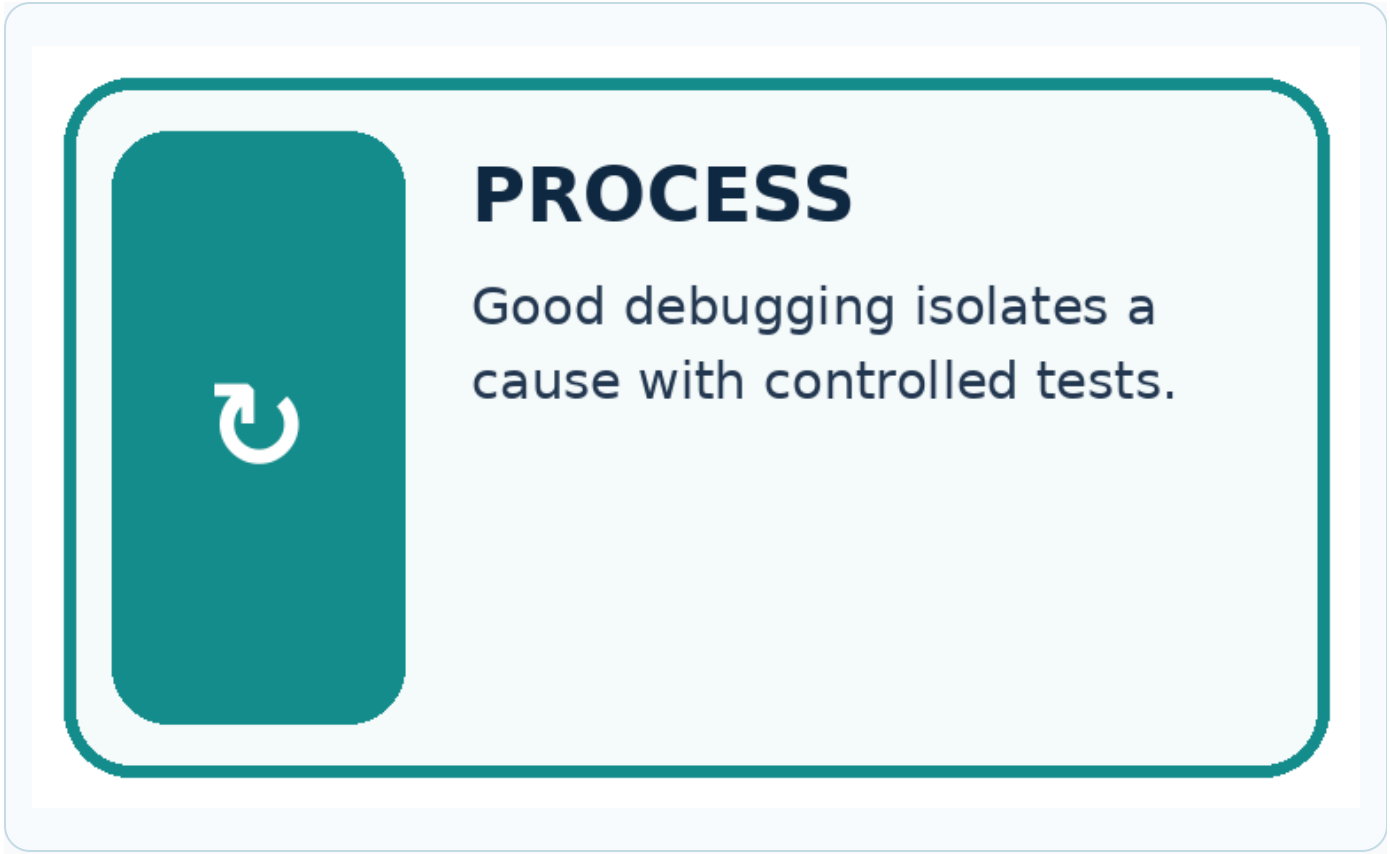
VISUAL **MECHANISM**

Input → **Process** → **Output**

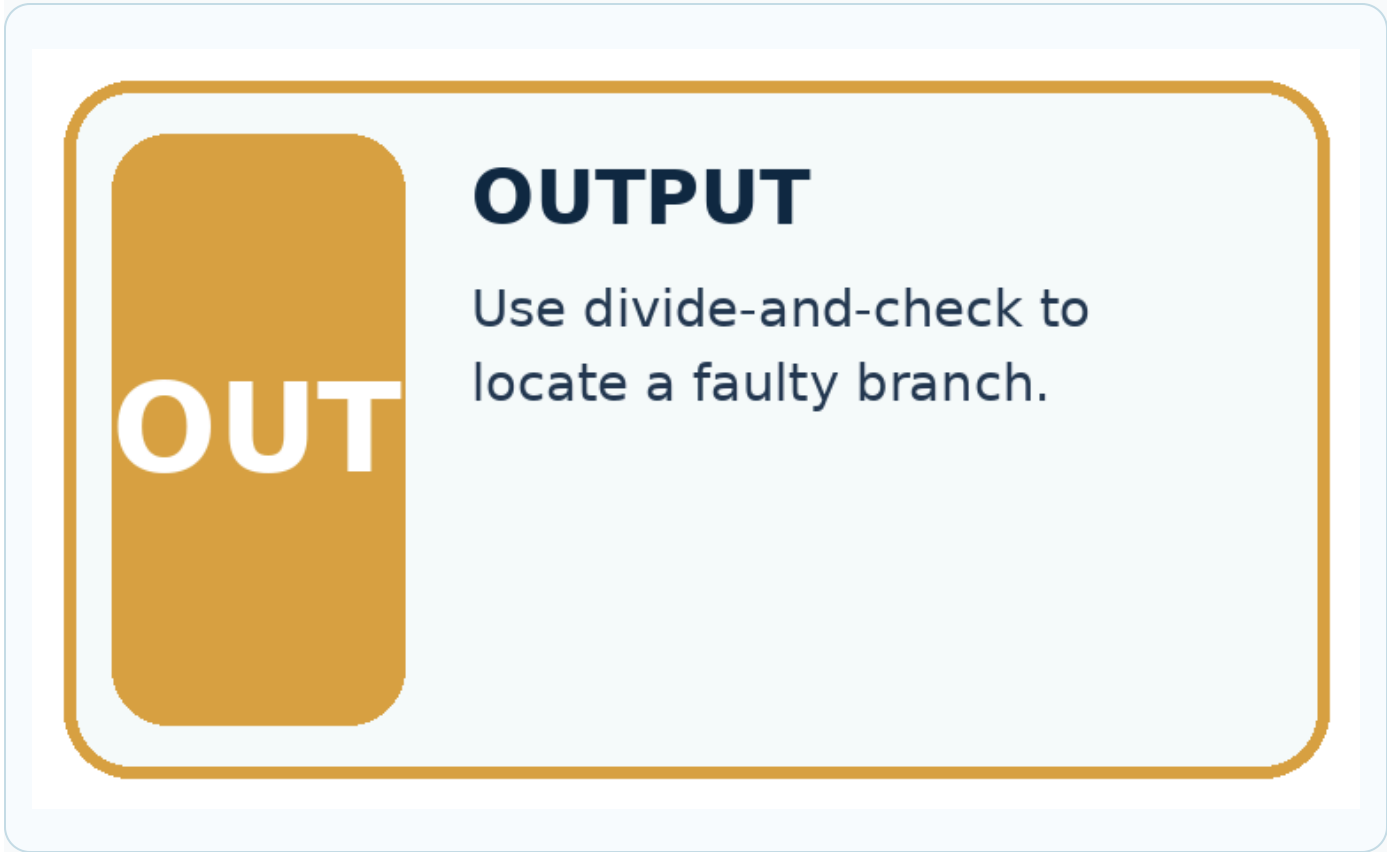
Trace the information. At each stage, ask what changed and how you could test it.



↓


PROCESS
Good debugging isolates a cause with controlled tests.

↓


OUT
OUTPUT
Use divide-and-check to locate a faulty branch.

Which stage could fail even when the other two are correct? Explain:

> **Write or draw your reasoning and evidence.**

A closer explanation

Give each test a job

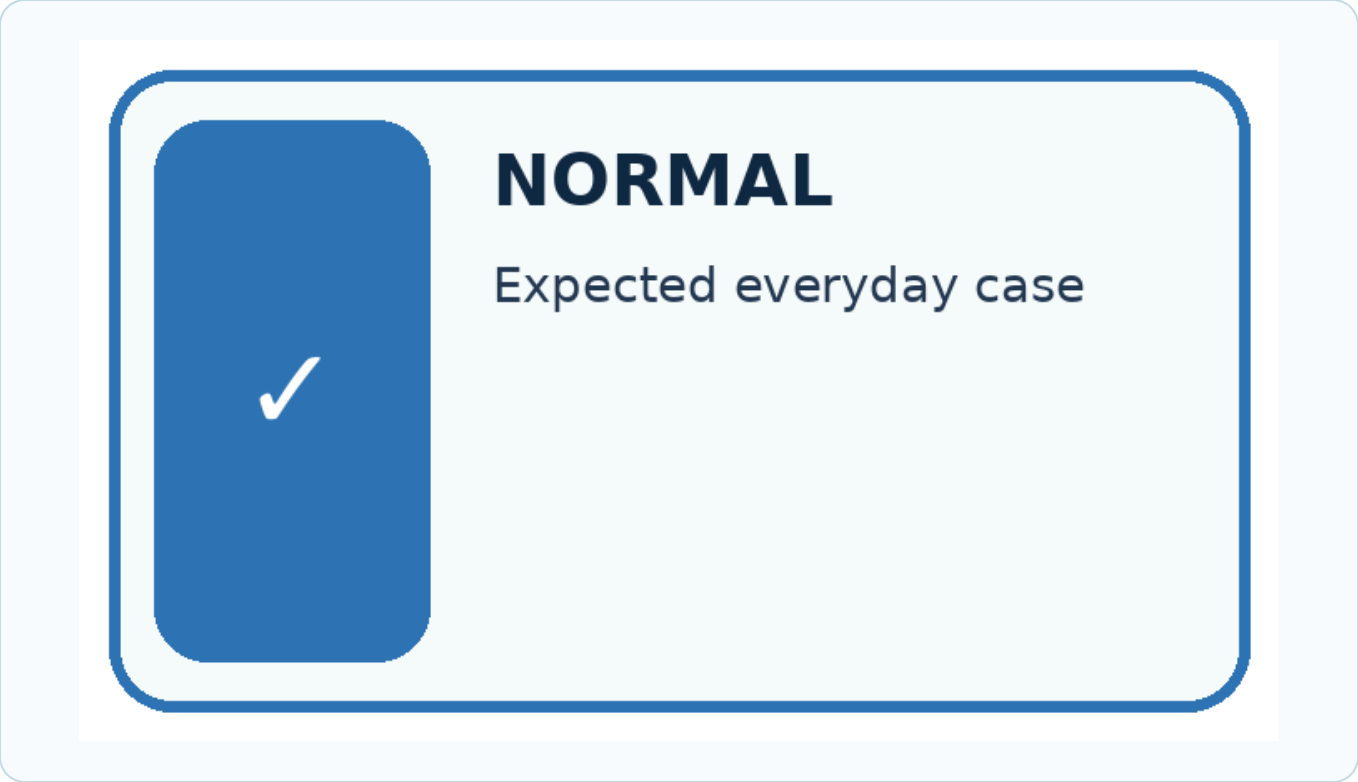
Connect the picture and the explanation

MAIN LESSON · SECTION 4

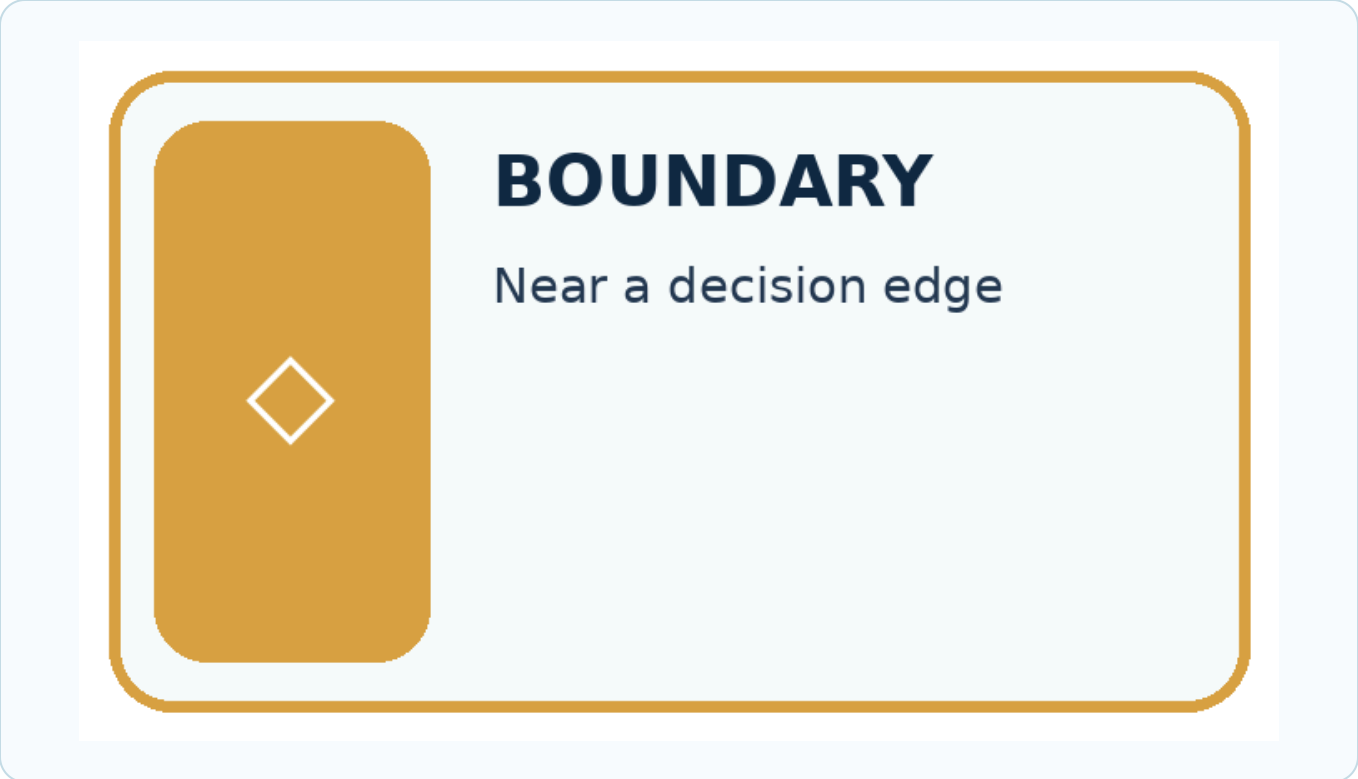
Compare ordinary, boundary, zero and transfer tests

> A revealing case depends on the specific suspected error.

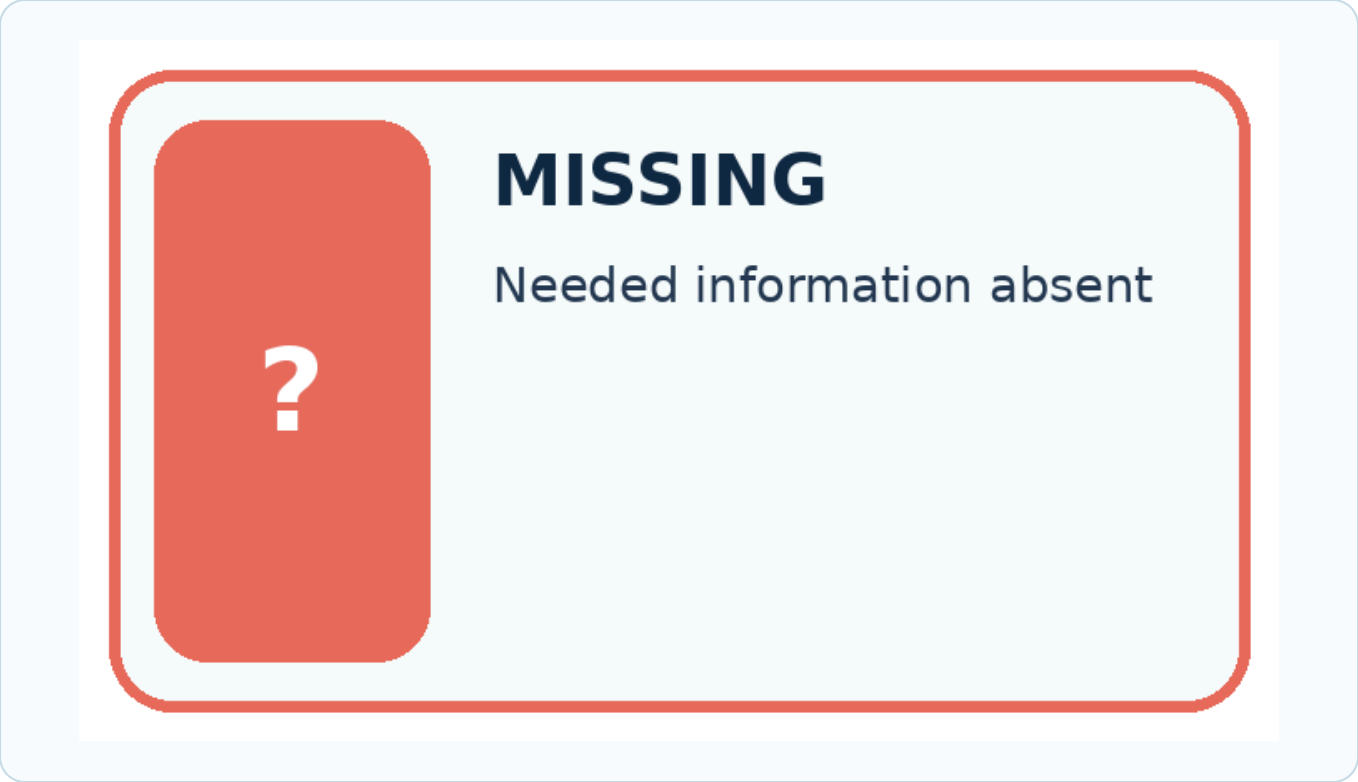
Four pictures with a purpose



Normal case: predict the result and deciding evidence.



Boundary case: identify the exact limit.



Missing information: name what must be clarified.



Transfer case: explain which procedure still applies.

Choose a revealing case

- Which case is most likely to reveal a hidden assumption? Why?

> **Write or draw your reasoning and evidence.**

Predict the supplied changed case

- Change: Repair the body to bundle_count*6+loose.

▼ Check after predicting

Prediction: Expected results for (2,1),(0,3),(2,0) are 13,3,12.

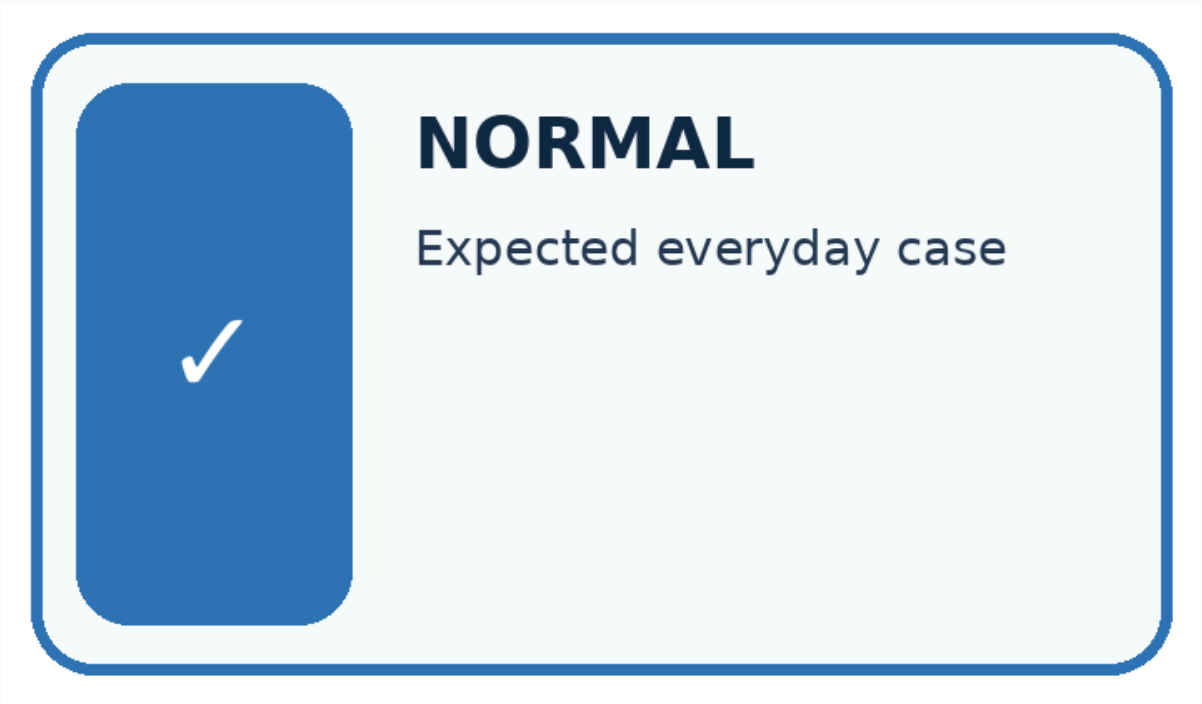
Reason: Only bundles are groups of six; loose books are counted individually.

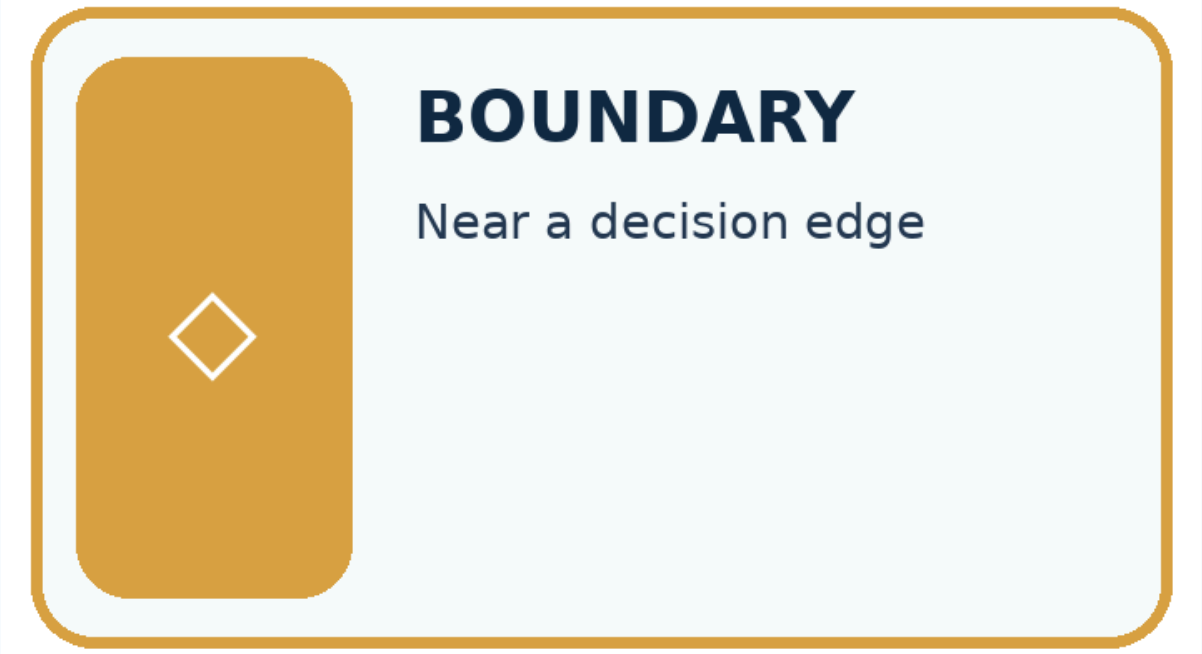
▼ Earlier wording, alternatives and picture transcripts

CASE GALLERY

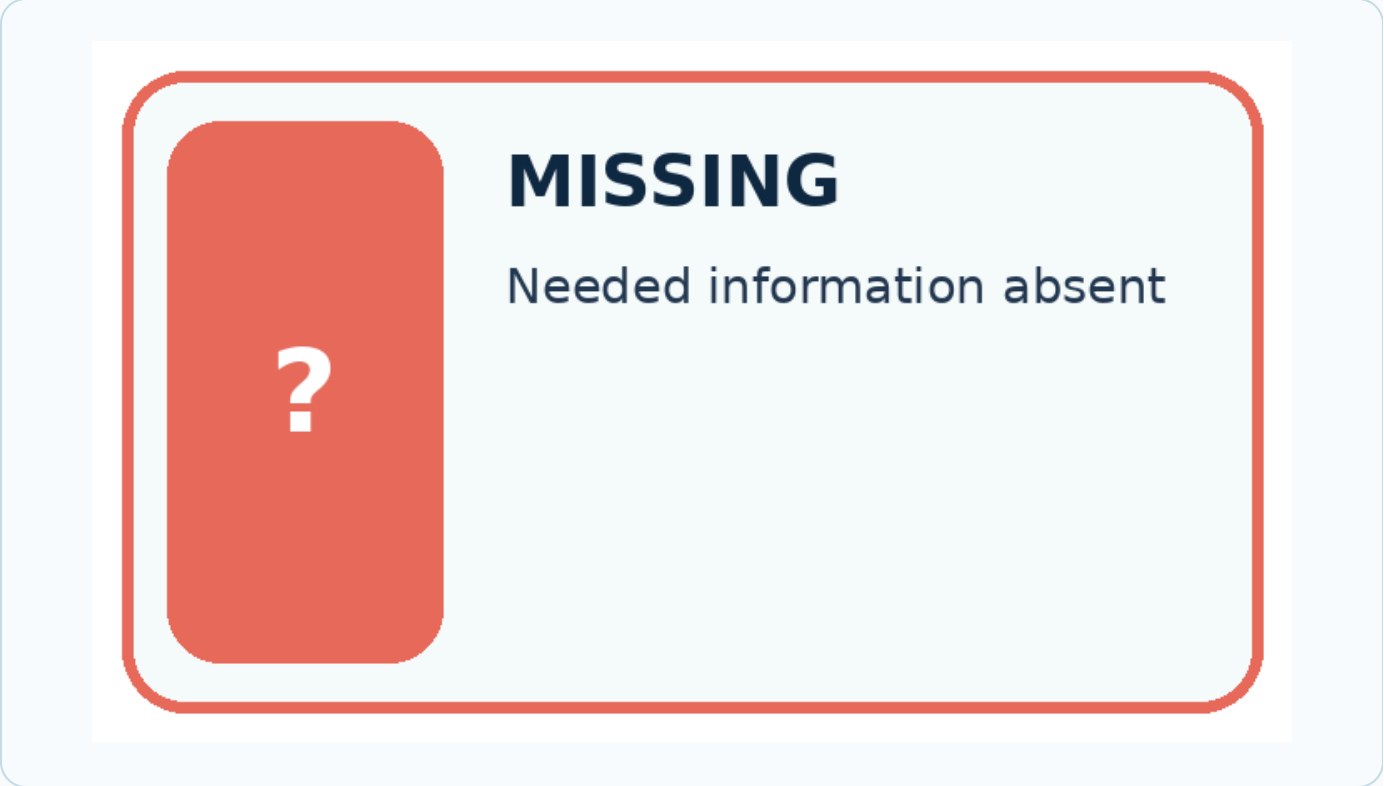
Four Tests, Four Kinds of **Evidence**

A system is not understood until it survives more than one comfortable example.





NORMAL CASE: predict and name **evidence**. **BOUNDARY** CASE: predict and name **evidence**.





MISSING INFORMATION: predict and name **evidence**. TRANSFER CASE: predict and name **evidence**.

Change one thing in the pictured case

MAIN LESSON · SECTION 6

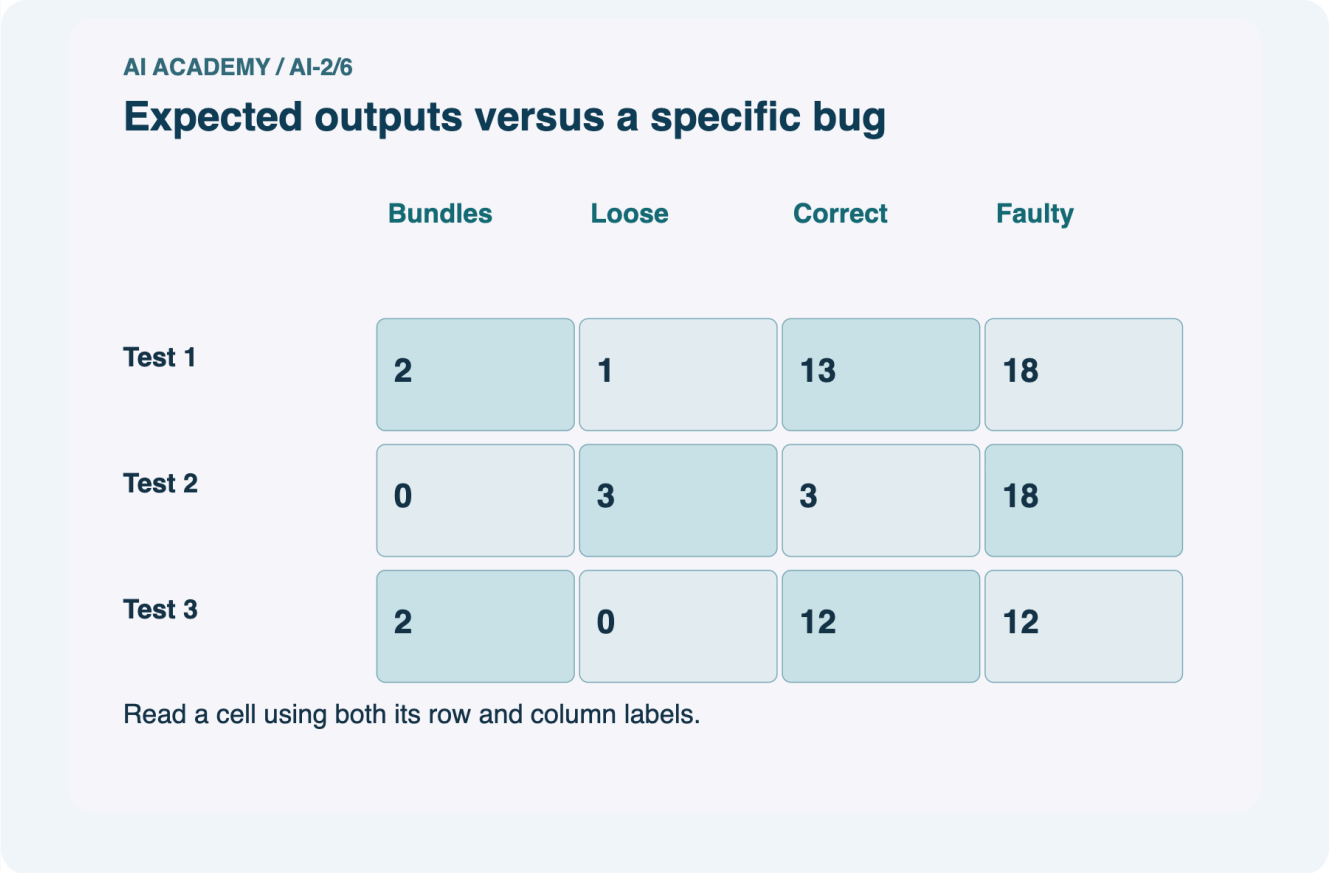
Expose constant answers, then the separate shelf formula bug

> Keep subtraction tests distinct from grouped bundles plus loose books.

Challenge a constant return value

Would replacing the body with return 5 pass every worked case? Identify the first revealing test.

Return to the main subtraction-test diagram



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the separate shelf rule and faulty grouping

A fictional shelf function should count books in complete bundles plus loose books.

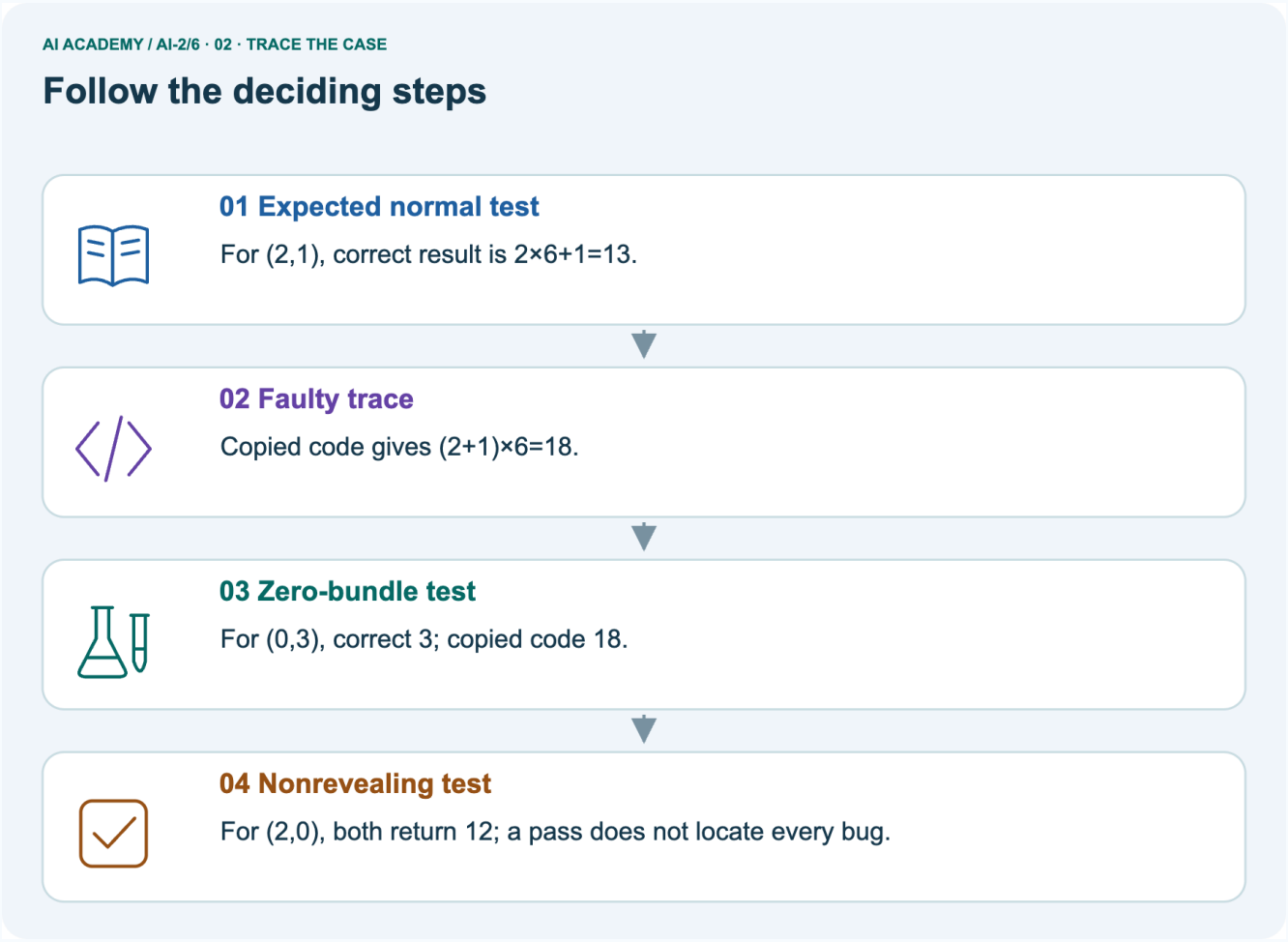
Correct definition: shelf(bundle_count,loose) returns bundle_count*6+loose.

Faulty copied body returns (bundle_count+loose)*6. Predict tests before running.

```
def shelf(bundle_count, loose):
    return (bundle_count + loose) * 6

for bundles, loose in [(2, 1), (0, 3), (2, 0)]:
    print(shelf(bundles, loose))
```

Trace revealing and nonrevealing cases



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Expected normal test	For (2,1), correct result is 2×6+1=13.
2. Faulty trace	Copied code gives (2+1)×6=18.
3. Zero-bundle test	For (0,3), correct 3; copied code 18.
4. Nonrevealing test	For (2,0), both return 12; a pass does not locate every bug.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

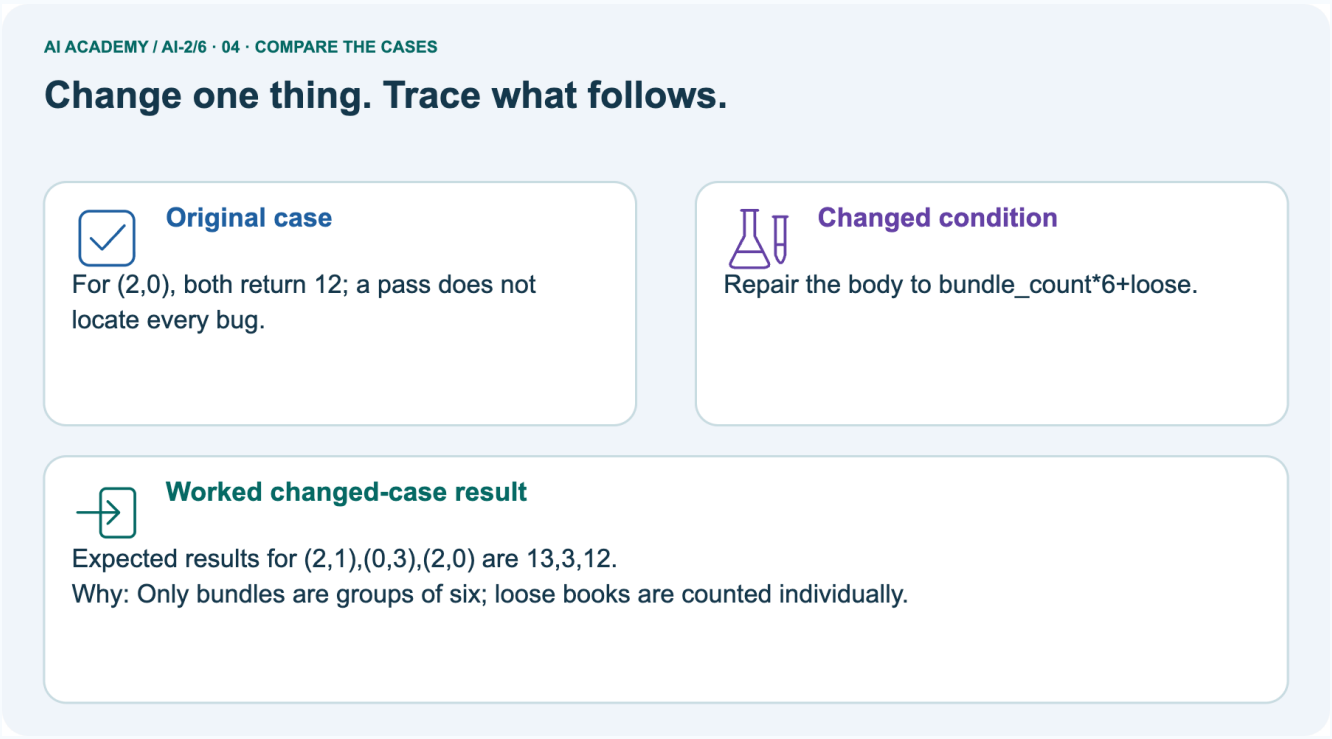
Test cases need not all expose the same error. Here loose books must be added after multiplying bundles. A zero-loose case cannot distinguish the two formulas because multiplying zero loose books changes nothing.

Keep expected and actual columns so an observed **output** does not quietly become its own expected answer.

Predict the targeted expression repair

- Before opening the repaired-shelf picture, change only the faulty body to bundle_count*6+loose. Predict (2,1),(0,3),(2,0). Which previously passing case must stay correct?

▼ Check the retained regression case



Follow the labelled picture with the matching explanation. Open the vector diagram

Explain the loose-book test and input limit

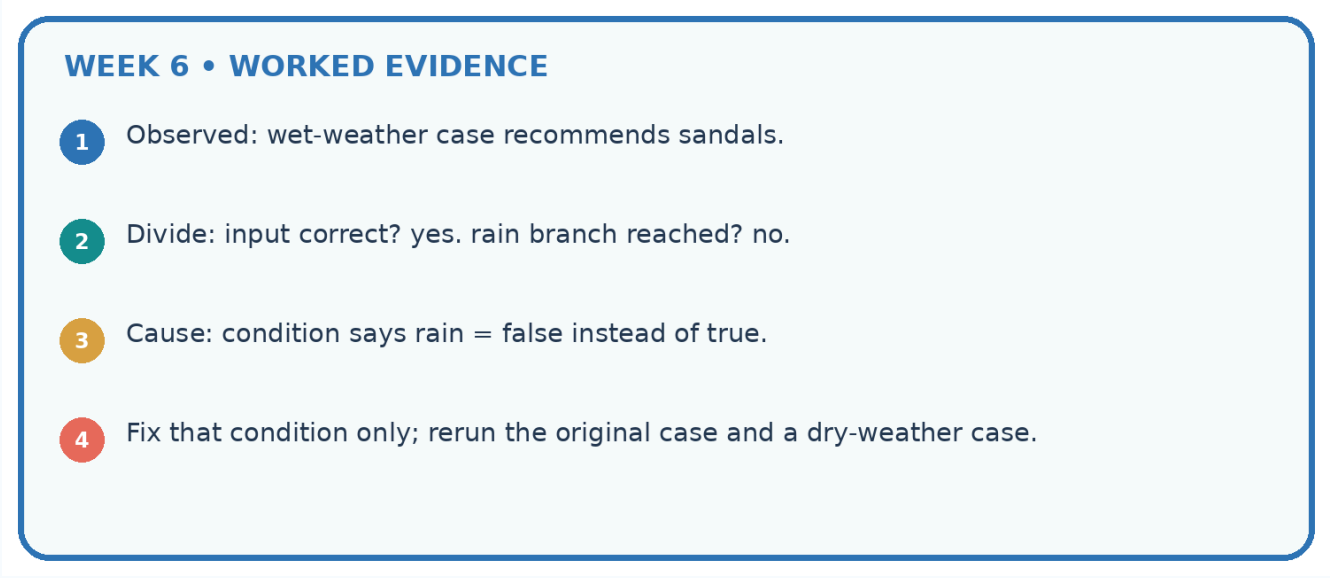
- Read the labelled picture: Which test most directly isolates the handling of loose books, and what should it produce?
- These three tests check a specified formula and bug; they do not prove correct handling of every type or invalid request.

▼ Earlier wording, alternatives and picture transcripts

WORKED **EVIDENCE**

Reason Step by Step

Predict first. Then check each step. A correct answer without visible reasoning is incomplete **evidence**.



1. Observed: wet-weather case recommends sandals.

2. Divide: **input** correct? yes. rain branch reached? no.

3. Cause: condition says rain = false instead of true.

4. Fix that condition only; rerun the original case and a dry-weather case.

Recalculate, retrace, or restate the reasoning in your own representation:

> **Write or draw your reasoning and evidence.**

CHECK Did the **evidence** answer the exact question? Did the total, path, or source record stay consistent?

A closer explanation

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Debugging Strategies

The facts we will use

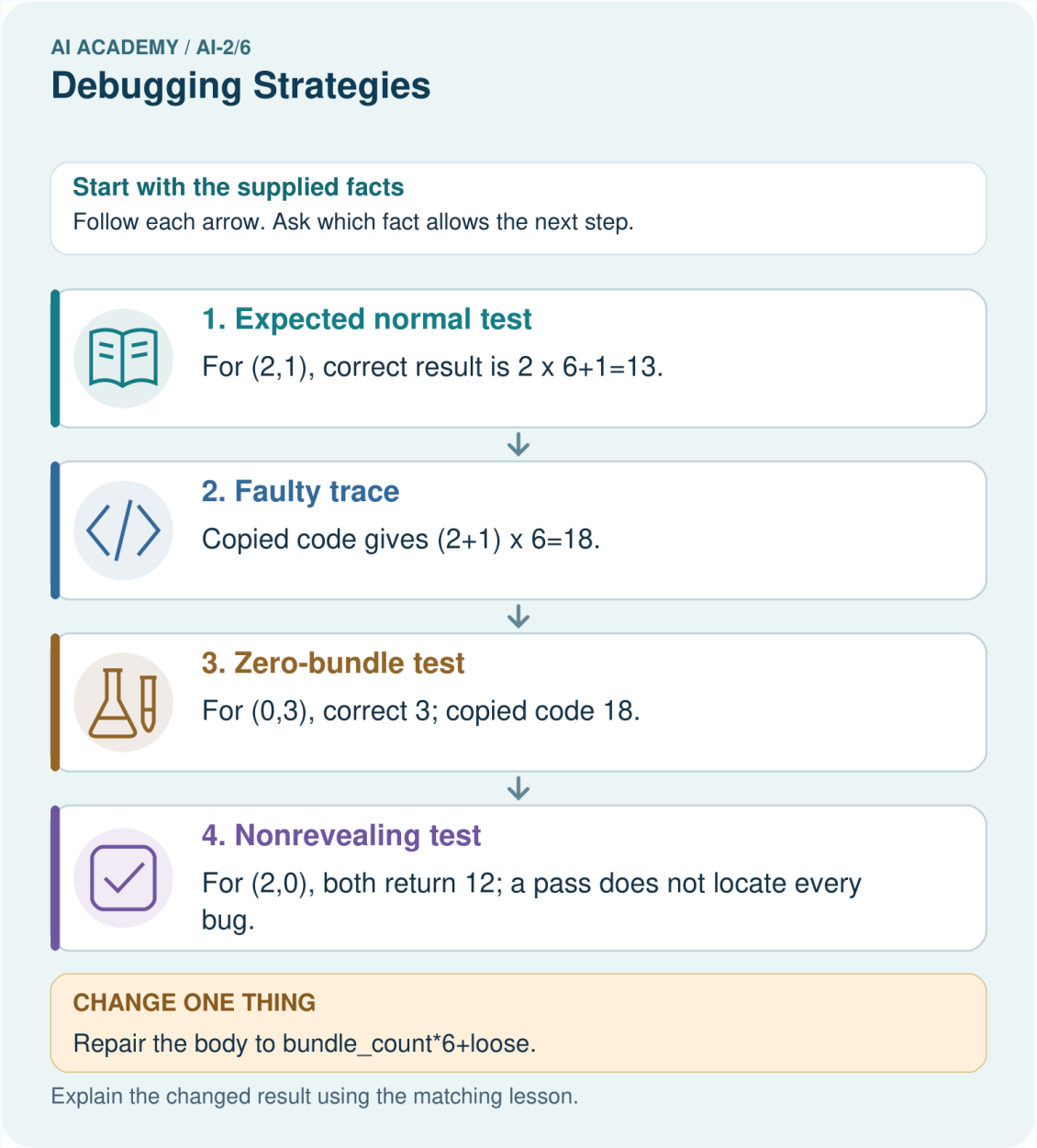
AI ACADEMY / AI-2/6

Expected outputs versus a specific bug

	Bundles	Loose	Correct	Faulty
Test 1	2	1	13	18
Test 2	0	3	3	18
Test 3	2	0	12	12

Read a cell using both its row and column labels.

Follow the labelled picture with the matching explanation. Open the vector diagram



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

Change one thing: Repair the body to bundle_count*6+loose. Predict the result before checking the explanation. What remains the same?

Limit the conclusion to what the suite actually checked

> A few valid-input passes do not establish invalid-input behaviour.

Read the suite's evidence limit

Passing this small suite supports these particular cases. It neither proves every allowed **input** correct nor checks invalid **input** handling. A test outside the contract needs a separately specified expected behavior.

Challenge the negative-count claim

> **All three stock tests pass. Does that prove correct handling of negative counts?**

▼ **Earlier wording, alternatives and picture transcripts**

ERROR DETECTIVE

Compare → Diagnose → Repair

Do not patch the answer. Find the earliest unsupported assumption, wrong step, or weak **evidence**.

!

COMPARE • DIAGNOSE

Weak idea: Changing several steps at once makes debugging faster. Strong move: locate evidence, change one cause, and retest.

Evidence record	Expected	Actual	Likely cause / next test
normal case	works		
boundary case	stated rule		
missing input	safe response		
fresh transfer	generalizes		

Repair one cause. Why is this repair better than changing several things?

Write or draw your reasoning and evidence.

A closer explanation

Practise: keep expectations fixed during repair

> A zero-used test can hide addition where subtraction is required.

Trace the faulty addition and targeted repair

A faulty return capacity + used gives 11, 16 and 8. The unused case passes even though the rule is wrong. Repair the operator to subtraction and rerun all three cases.

Never change an expected result merely to match faulty **output**.

Find a revealing case in W2

Use workbook W2 for the connected example “Testing and **debug**ging functions”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Repair the successful-run guarantee

- Use the worked case for Testing and **debug**ging functions. Proposed explanation: “A successful run proves the conclusion”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Return to the shelf’s loose-book check

Which test most directly isolates the handling of loose books, and what should it produce?

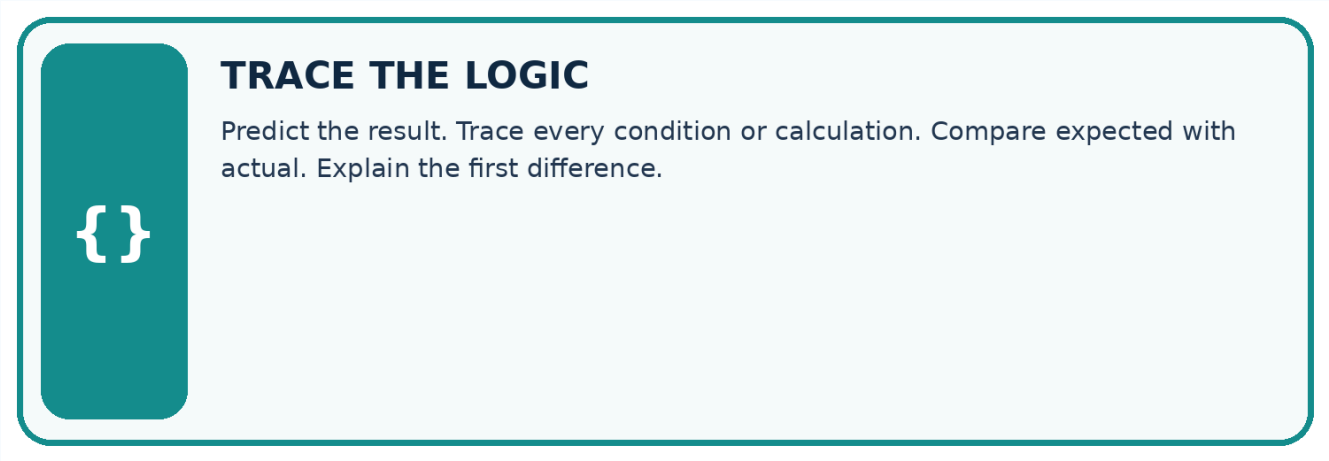
This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ Earlier wording, alternatives and picture transcripts

LOGIC STUDIO

Trace Before You Run

Block code, pseudocode, tables, and diagrams are different ways to expose the same reasoning.



SET **evidence**_record PREDICT expected_result RUN one normal case and one **boundary** case COMPARE actual_result with expected_result IF different: locate first cause; change one factor; **RETEST** EXPLAIN result, limitation, and next safe step

Trace one **input**. Which line changes the state or chooses a path?

Write or draw your reasoning and evidence.

A closer explanation

Practise with less help: Find a revealing case

Check your reasoning

Explain the picture

> **My explanation using a pictured step and the given facts**

MAIN LESSON · SECTION 9

Investigate three implementations under one fixed suite

> A controlled comparison changes the implementation, not the rule or expected answers.

The supplied suite and three candidate bodies

Use the supplied remaining function and its three tests (8,3→5),(8,8→0),(8,0→8). Before running, predict the actual values and Boolean checks for three labelled versions: correct capacity-used; faulty capacity+used; faulty constant 5. Change only the function body, keeping cases and expected values identical. Explain which test distinguishes each fault.

Predict, execute, repair and retain the full record

1. Run each saved version and record actual values as well as True/False comparisons; preserve errors and all results. Repair only the located expression, then rerun every case. Add the workbook's six-capacity cases with independent expectations and state what invalid inputs remain unspecified. If using assert, record that execution stops at the first failing assertion instead of treating later tests as passes. Label paper-only work traces and reserve stock(packs,loose) for independent transfer.

Keep the evidence table

Test / input · Prediction · Actual · Evidence / calculation · Change or keep?

> **Test / input:** normal
> **Prediction:** _____
> **Actual:** _____
> **Evidence / calculation:** _____
> **Change or keep?:** _____

> **Test / input:** boundary
> **Prediction:** _____
> **Actual:** _____
> **Evidence / calculation:** _____
> **Change or keep?:** _____

> **Test / input:** missing
> **Prediction:** _____
> **Actual:** _____
> **Evidence / calculation:** _____
> **Change or keep?:** _____

> **Test / input:** fresh transfer

> **Prediction:** _____
>
> **Actual:** _____
>
> **Evidence / calculation:** _____
>
> **Change or keep?:** _____

Explain the most useful case

- What did the hardest test teach you that the normal test did not?

- Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

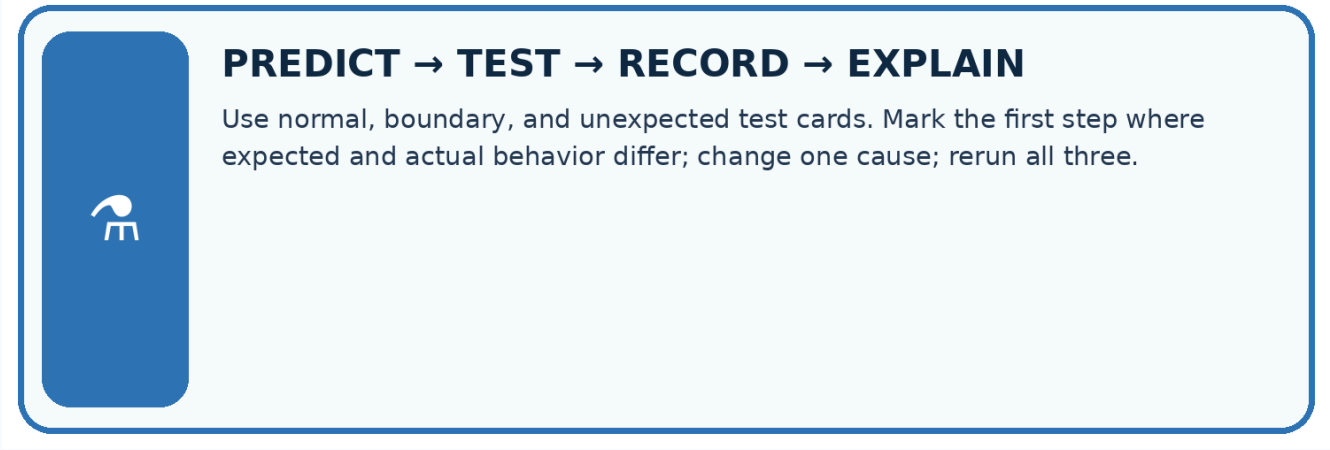
> **Write or draw your reasoning and evidence.**

▼ **Earlier wording, alternatives and picture transcripts**

EVIDENCE LAB

Predict → Test → Record → Explain

Keep expected and actual results separate. Surprises are useful data when recorded honestly.



Use normal, **boundary**, and unexpected test cards. Mark the first step where expected and actual behavior differ; change one cause; rerun all three.

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: test the different stock formula

> Use four counters per pack and loose counters before feedback.

The independent function and calls

Define stock(packs, loose) to return packs * 4 + loose. Create expected tests for (2, 3), (0, 3), (2, 0), then show which detects a faulty packs + loose body.

Keep your first attempt

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Your teacher has the independent answer and scoring criteria. Complete the matching workbook week to keep your practice **evidence** together.

Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ **Earlier wording, alternatives and picture transcripts**

Independent case: Test a new function independently

Week 6 Independent application

Close the worked explanation. Apply the idea to this different question without copying.

A loop should run four times but runs five. You change its colour and stopping condition at once. Which single change is relevant to test first?

My answer, with a trace, calculation or supporting **evidence**

Write or draw your reasoning and evidence.

Create and test

Change one relevant detail. Predict the result before testing. Include a **boundary** or missing-information case when it helps reveal a limit.

My changed **input** and expected result

> **Write or draw your reasoning and evidence.**

Connect to your project

When a ranking surprises you, inspect feature values, weights, arithmetic, and sorting separately.

The **evidence** I will keep

> **Write or draw your reasoning and evidence.**

Transfer practice from the original programme

Intended score is 3 per matching trait. Bug: score starts at 3 and adds 3 for each match. Give expected and actual results for 0,1,2 matches and repair the cause.

> **Explain the mechanism, evidence and one limit.**

Use feedback to improve

MAIN LESSON · SECTION 11

Add revealing helper-function tests to the portfolio

> Make each test target an explicit project rule.

Scope of this week’s contribution

The Transparent Book Recommender’s stages need separate tests before a complete ranking can be trusted. Use fictional book records, declared scoring and eligibility rules, and expected values calculated independently. Inspect feature inputs, arithmetic and ordering separately; do not invent a score to make an observed ranking look correct.

The checkpoint and system map

THIS WEEK’S CHECKPOINT When a ranking surprises you, inspect feature values, weights, arithmetic, and sorting separately.

SYSTEM MAP Reader need → allowed **input**s → transparent rule or score → ranked candidates → explanation → test record → **human decision**

Test the recommender’s helper functions with normal, zero and **boundary input**s before **debug**ging the whole application.

Record the cause, repair and human choice

- What will you add, change, or verify in the project this week?
- What should remain under **human control**?

> **Write or draw your reasoning and evidence.**

Write or draw your reasoning and evidence.

Retain privacy and ranking limits

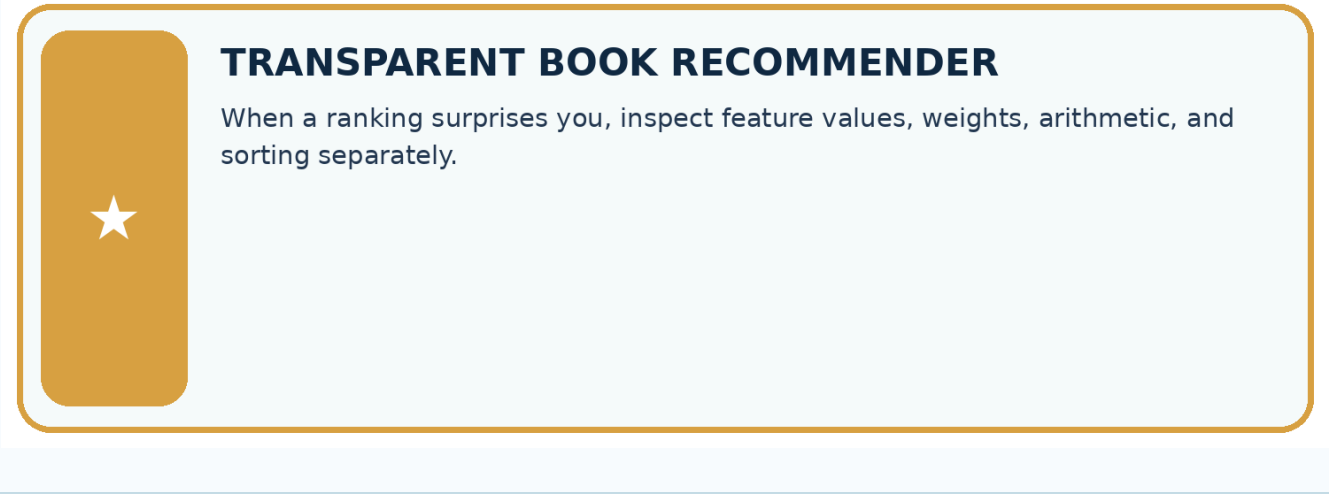
RESPONSIBLE DESIGN Collect only necessary information. Show uncertainty and limits.
Never confuse a ranking with a fact about a person.

▼ Earlier wording, alternatives and picture transcripts

CUMULATIVE PROJECT

Transparent Book Recommender

Every week adds one inspectable piece. The system must show how it reached a result.



Evidence for your project

Finish: explain why the test reveals the defect

> Close your notes and distinguish execution, correctness and tested scope.

Your exit explanation

- Close your notes. Explain this goal in your own words: Design revealing tests for a function

> **My deciding step, evidence and one remaining question**

Choose the next useful regression case

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Your quick word guide**

REFERENCE · SECTION 5

Your quick word guide

> Use the words to explain a particular step, record or limit.

Core vocabulary

Word	My own example	Not this / boundary
bug	A defect that makes a program behave differently from its intended specification.	
debug	Find and repair the cause of a program's incorrect behaviour.	
controlled test	A test with specified inputs, conditions and expected measurements.	
retest	Run a test again after a change or under specified new conditions.	

Terms for the connected explanation

Word	Meaning
Test case	Inputs and an expected result
Boundary	An edge of a permitted range

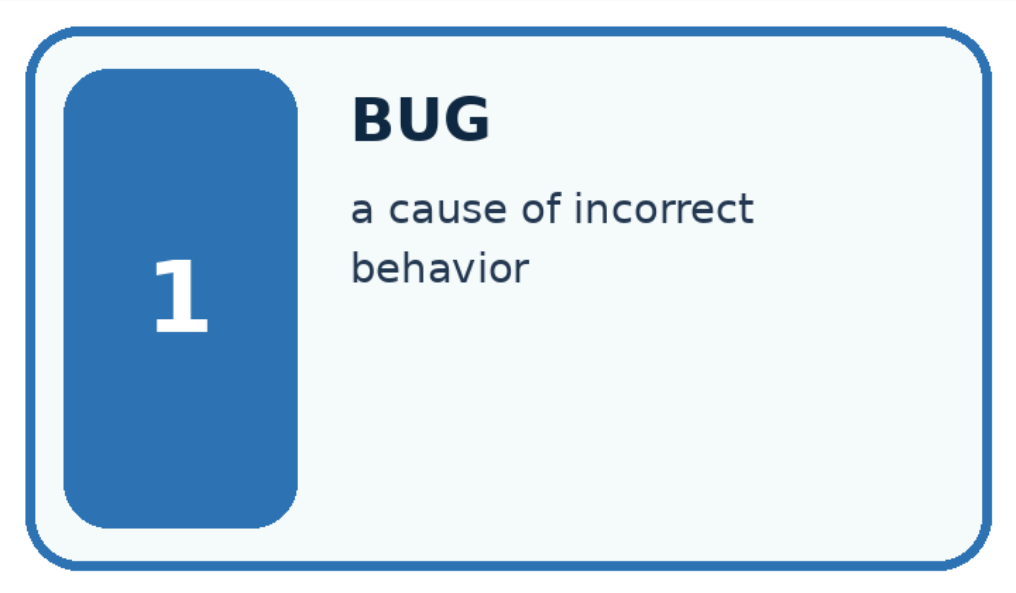
Word	Meaning
Regression	A previously working case that fails after a change
Assertion	A check that raises an error when its condition is false

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

VOCABULARY LAB

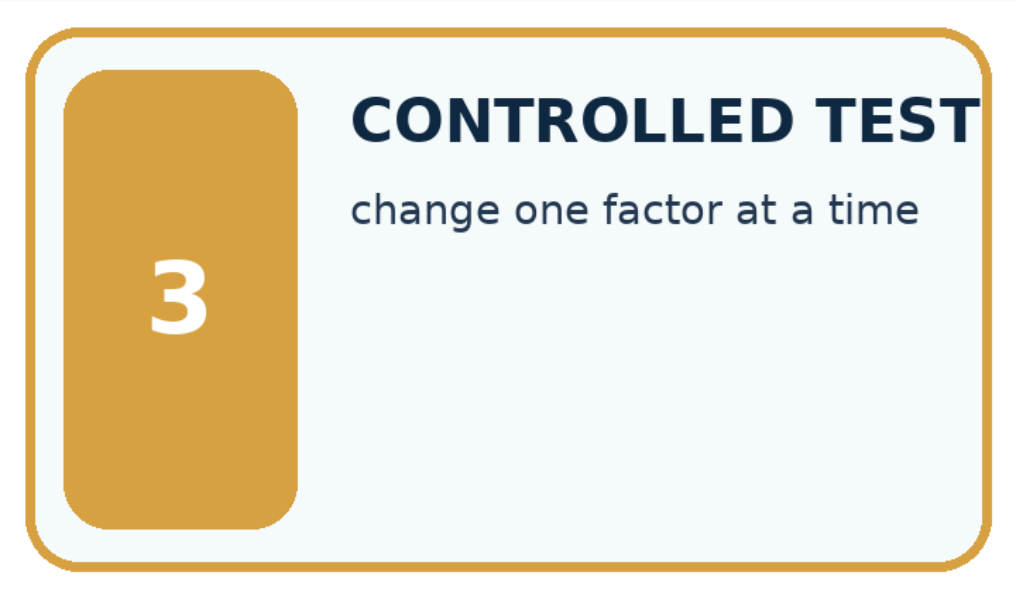
Words You Can Use to Reason

A definition matters when you can recognize the idea, use it, and contrast it with a near-miss.



</details>


2
DEBUG
find and repair a cause


3
CONTROLLED TEST
change one factor at a time



Words used in the closer explanation

▼ **Choose a stronger thinking challenge**

REFERENCE · SECTION 10

Choose a stronger thinking challenge

> Choose the next useful challenge rather than treating the whole ladder as compulsory.

Pick your next step

- 1 • NAME: Define the concept without repeating the textbook.
- 2 • TRACE: Show the information, state, branch, calculation, or **evidence** path.
- 3 • APPLY: Solve a different example independently.
- 4 • CHALLENGE: Find a **boundary** case, counterexample, or unfair outcome.

- 5 • EXPLAIN LIMITS: Say when the result should not be trusted or used alone.

- My strongest **evidence** of independent understanding:

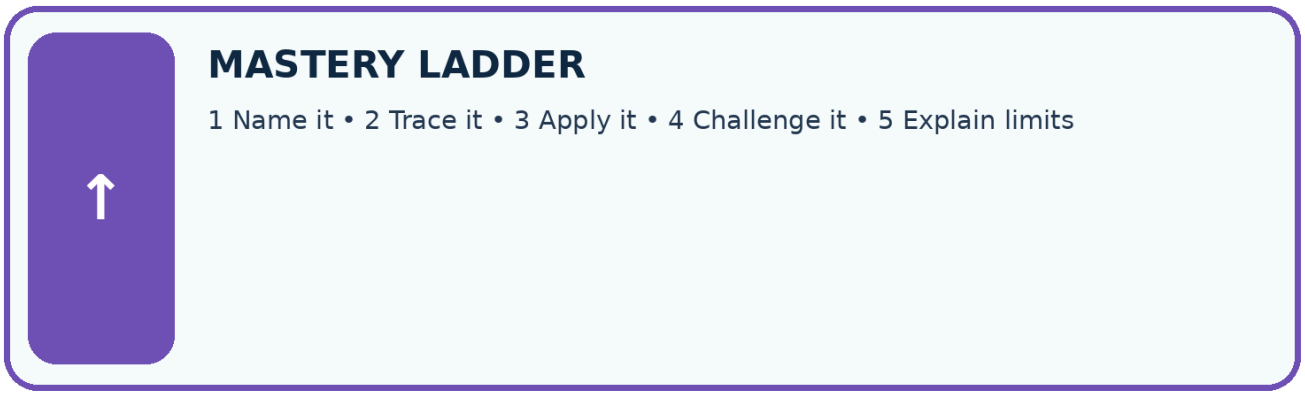
Write or draw your reasoning and evidence.

▼ Earlier wording, alternatives and picture transcripts

MASTERY

From Knowing the Word to Owning the Idea

Move upward only when you can produce **evidence** without copying the worked example.



▼ Reference: check your explanation

Reference: check your explanation

> Use after practice. Test a new function independently is the selected independent case.

Check the evidence and its limit

- 1. My fresh example, trace, calculation, or design:
- 2. My **evidence** that it works (include a **boundary** or missing-data case):
- 3. One limitation, fairness concern, or situation needing human review:

> **Write or draw your reasoning and evidence.**

> **Write or draw your reasoning and evidence.**

Self-check and record

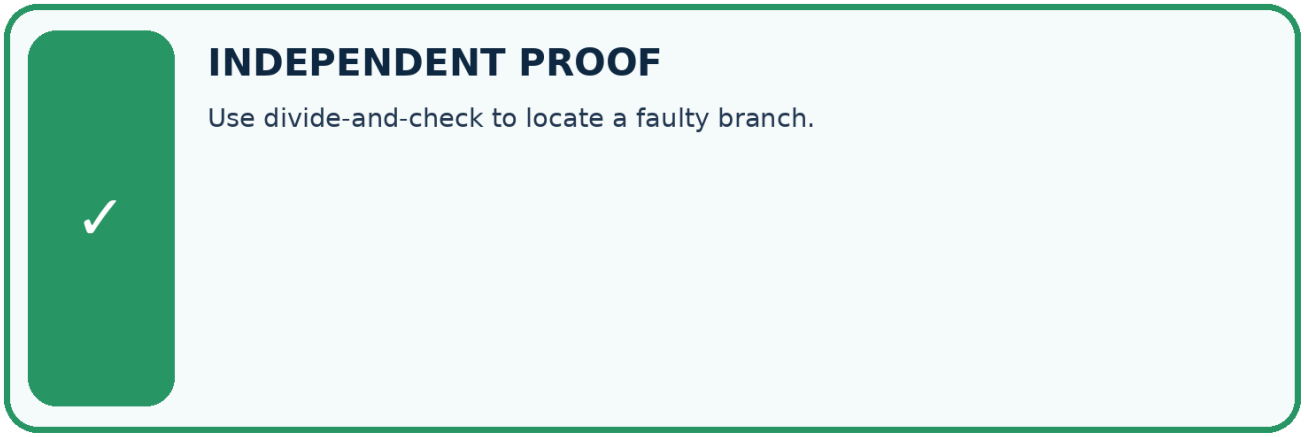
SELF-CHECK ☐ I explained the **mechanism** ☐ I used **evidence** ☐ I tested a fresh case ☐ I named a limitation ☐ I can answer 'why?'

▼ **Earlier wording, alternatives and picture transcripts**

INDEPENDENT MASTERY PROOF

Checklist for explaining this week's idea

Use after practice. Test a new function independently is the selected independent case.



EXPLANATION CHECK Keep the rule and expected values fixed, choose calls that distinguish a defect, and rerun earlier cases after a targeted repair.

Write or draw your reasoning and evidence.

▼ **Extra practice: Put the addition inside the condition**

REFERENCE · SECTION 13

Extra practice: Put the addition inside the condition

> Try the question before opening its worked explanation.

Read the supplied case and question

Award two points per matching trait. A bug awards two for every trait.

Test one match and one nonmatch.

Your explanation

- Which detail supports the answer, and why does it matter?

Write or draw your reasoning and evidence.

▼ Check after trying

Expected 2; buggy result 4. Put the addition inside the match condition. Test zero, one and two matches for 0,2,4.

Evidence or stage	Value or result
One matching trait	+2
One nonmatching trait	+0
Expected total	2
Buggy total	4 because both were awarded points

▼ Earlier wording, alternatives and picture transcripts

Week 6 Worked case

Use the supplied details to practise the weekly concept before attempting the independent question. This case extends the illustrated lesson.

The situation

The question

Worked explanation

Explain it in your own words

Trace the supplied case visually

Put the addition inside the condition

Award two points per matching trait. A bug awards two for every trait.

Test one match and one nonmatch.

Expected 2; buggy result 4. Put the addition inside the match condition. Test zero, one and two matches for 0,2,4.

▼ Reference: original lesson checklist

REFERENCE · SECTION 15

Reference: original lesson checklist

> Use an additional checklist when it helps inspect your explanation.

Targets and evidence

By the end, I can	How I will show it
Design revealing tests for a function	Explain the deciding step, show a trace or test, and state one limit.
Compare expected and actual results	Explain the deciding step, show a trace or test, and state one limit.
Repair a defect and rerun earlier cases	Explain the deciding step, show a trace or test, and state one limit.

Keep your route and first prediction

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Earlier wording, alternatives and picture transcripts

Start with this question: A fictional shelf function should count books in complete bundles plus loose books.

▼ Extra practice: pictures, repair and feedback

REFERENCE · SECTION 17

Extra practice: pictures, repair and feedback

> Attempt each question before opening its explanation. Use one browser or paper copy.

First supported question

- A fictional shelf function should count books in complete bundles plus loose books.

- Which labelled step matches this detail? For (2,0), both return 12; a pass does not locate every bug.
- 1. Zero-bundle test
- 2. Expected normal test
- 3. Faulty trace
- 4. Nonrevealing test
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**
>
> ---
>
> ---
>
> ---
>
> ---

▼ **Check the first explanation**

After your attempt, compare with the supported explanation: Nonrevealing test For (2,0), both return 12; a pass does not locate every bug. **Test case**s need not all expose the same error.

Here loose books must be added after multiplying bundles. A zero-loose case cannot distinguish the two formulas because multiplying zero loose books changes nothing.

Keep expected and actual columns so an observed **output** does not quietly become its own expected answer.

Changed-case question

- Changed-case practice: Repair the body to bundle_count*6+loose. Which conclusion is supported by the supplied facts?
- 1. We can decide the result without examining the changed **input** or the procedure.
- 2. Expected results for (2,1),(0,3),(2,0) are 13,3,12.
- 3. This one example guarantees correct results for every future **input**.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check the changed case**

After your attempt, compare with the supported explanation: Expected results for (2,1),(0,3), (2,0) are 13,3,12. Only bundles are groups of six; loose books are counted individually.

Repair and save your work

Review Loops and Stopping (ai-2/5). Goal: Trace variable values before and after each loop iteration. Short practice: Why is report 0 correct for the original loop, and why is equality-only stopping risky here?.

Return to this week and explain what you changed.

Review Branching Logic (ai-2/3). Goal: Trace the selected branch in an if elif else chain. Short practice: Give expected labels for –1,0,10,11 and identify the equality **boundary** for Short..

Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work
</details>

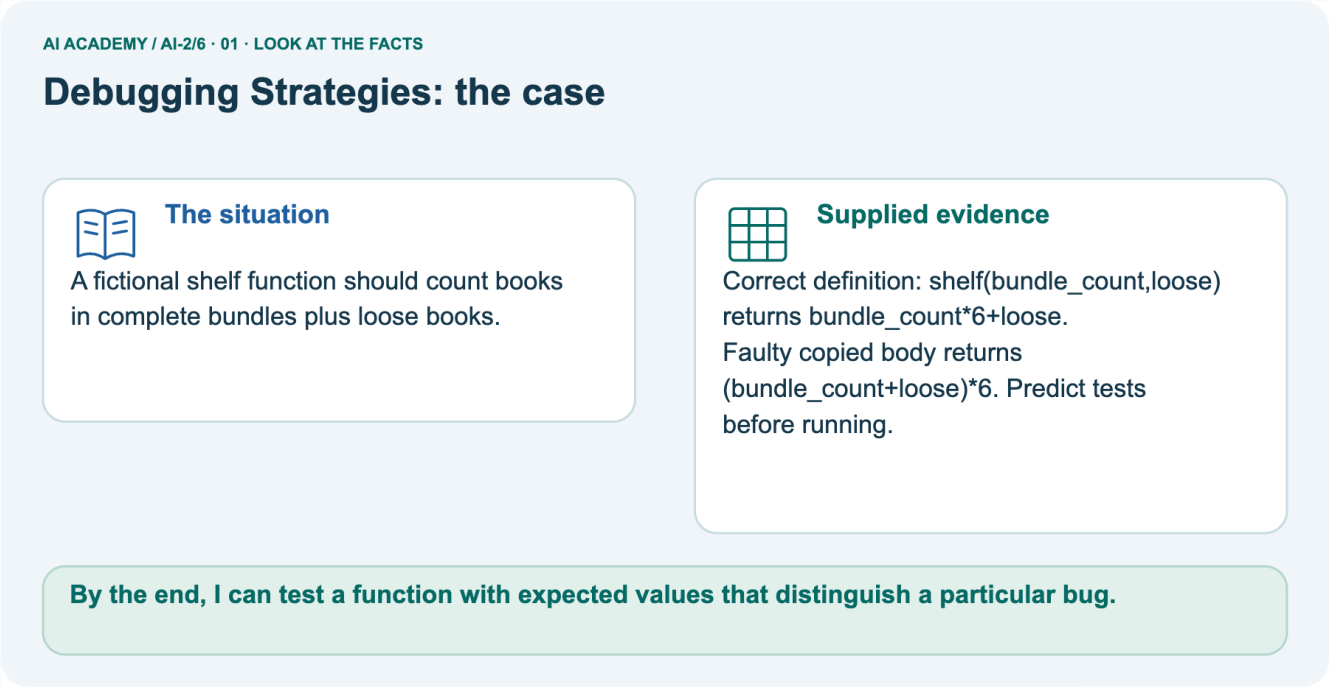
<details>
<summary>Reference: the six-picture journey</summary>
</details>

REFERENCE · SECTION 18

Reference: the six-picture journey

> Revisit the picture that serves the current question; key diagrams already sit beside the main explanation.

01. Debugging Strategies: the case



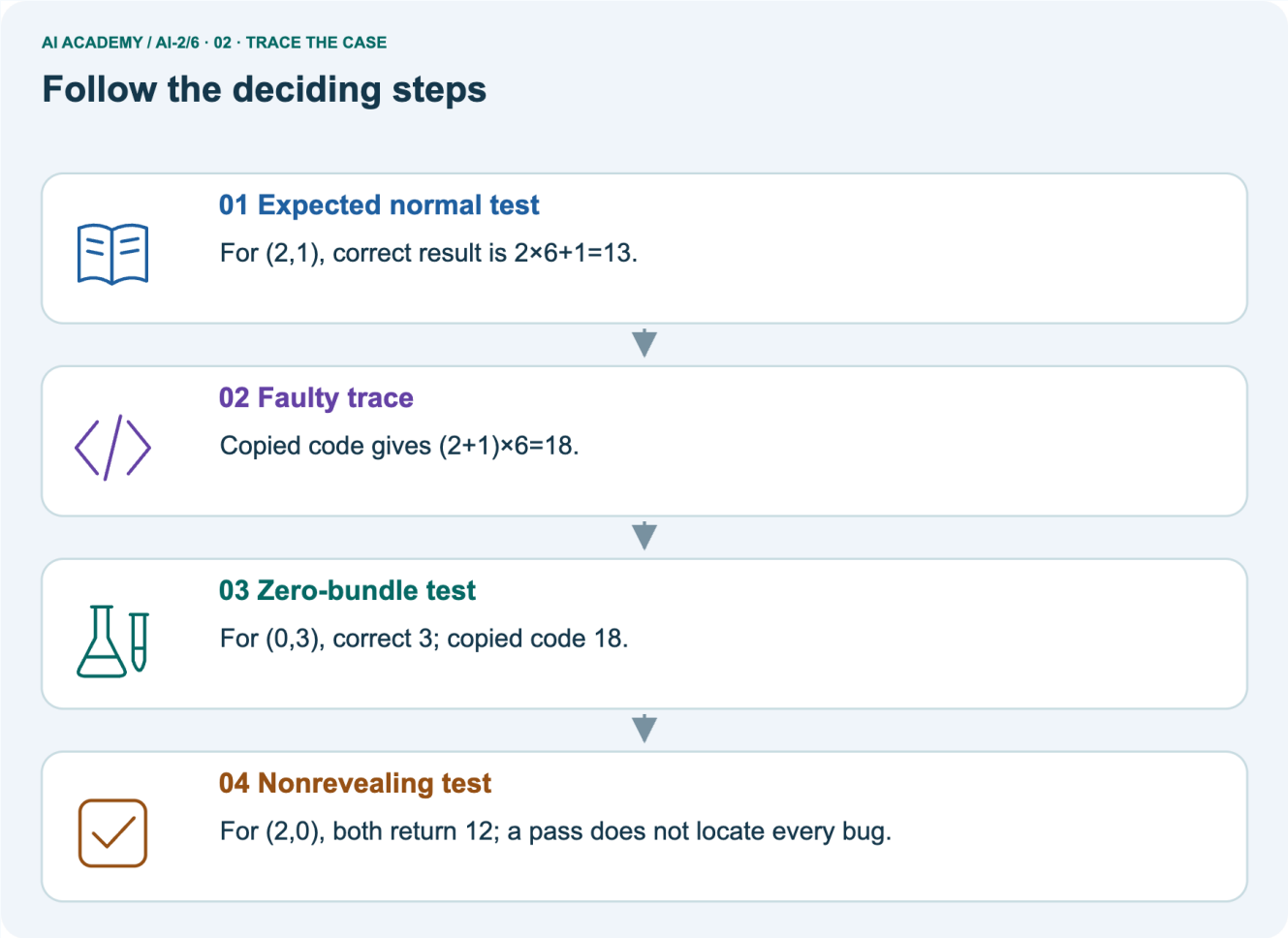
Follow the labelled picture with the matching explanation. Open the vector diagram

A fictional shelf function should count books in complete bundles plus loose books. Supplied facts: Correct definition: shelf(bundle_count,loose) returns bundle_count*6+loose. Faulty copied body returns (bundle_count+loose)*6.

Predict tests before running.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps

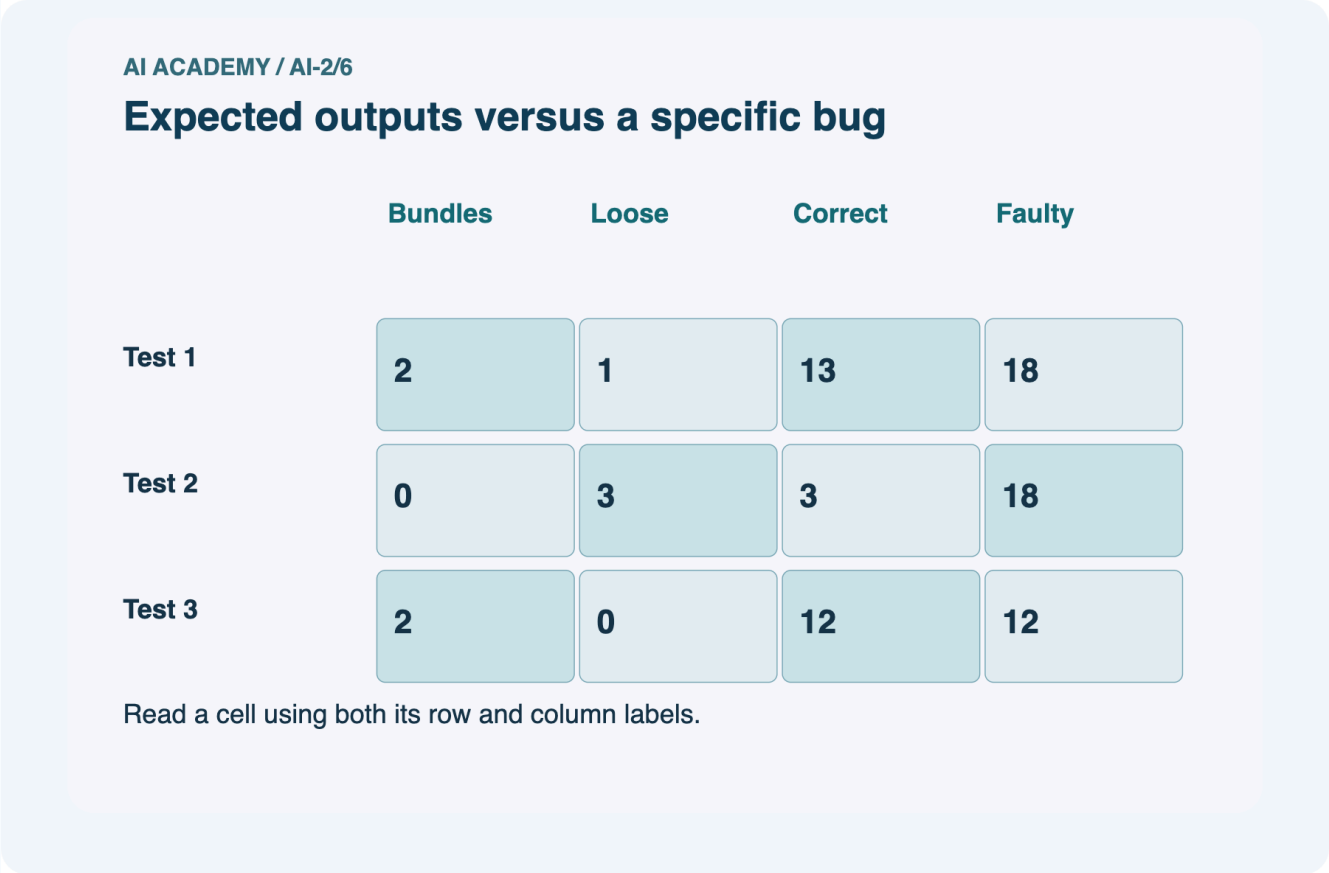


Follow the labelled picture with the matching explanation. Open the vector diagram

Expected normal test: For (2,1), correct result is 2×6+1=13. → Faulty trace: Copied code gives (2+1)×6=18. → Zero-bundle test: For (0,3), correct 3; copied code 18. → Nonrevealing test: For (2,0), both return 12; a pass does not locate every bug.

Point to each numbered stage and explain which recorded **input** it uses.

03. Expected outputs versus a specific bug

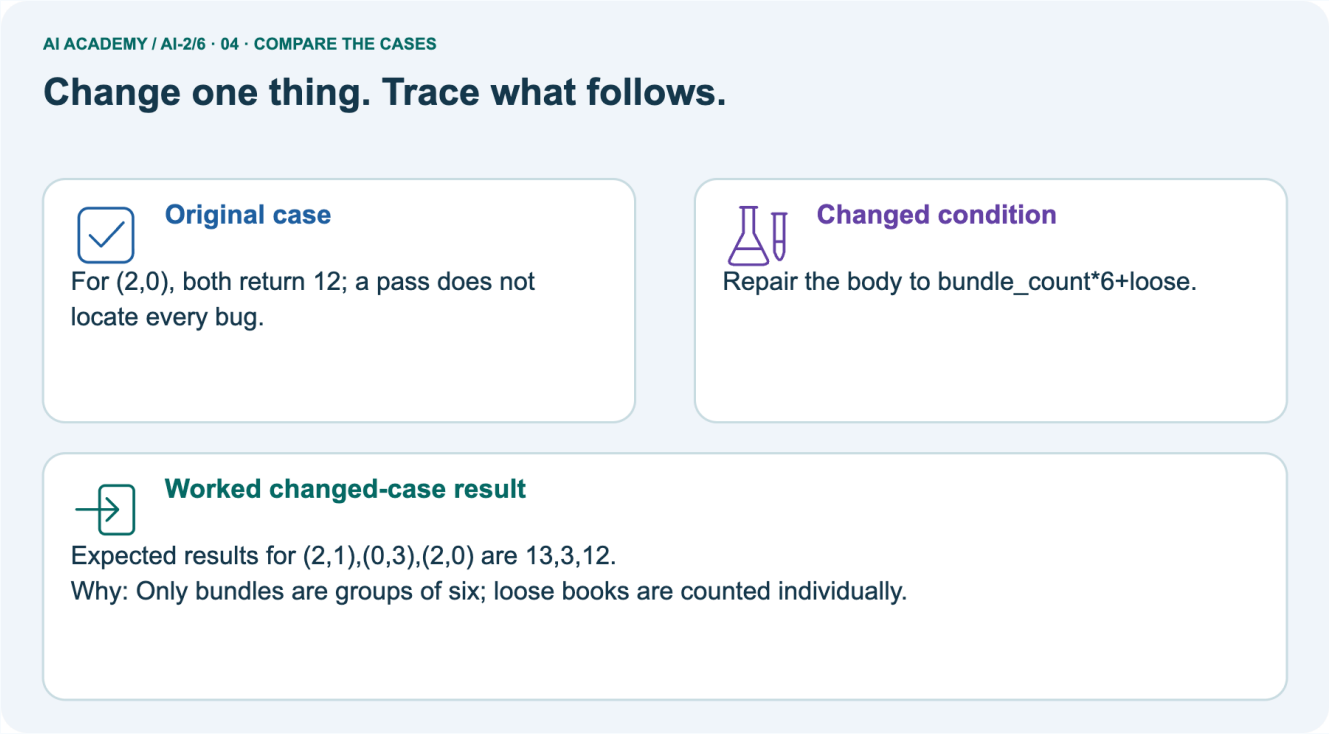


Follow the labelled picture with the matching explanation. Open the vector diagram

Expected **output**s versus a specific bug. Test 1: Bundles = 2, Loose = 1, Correct = 13, Faulty = 18. Test 2: Bundles = 0, Loose = 3, Correct = 3, Faulty = 18. Test 3: Bundles = 2, Loose = 0, Correct = 12, Faulty = 12.

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



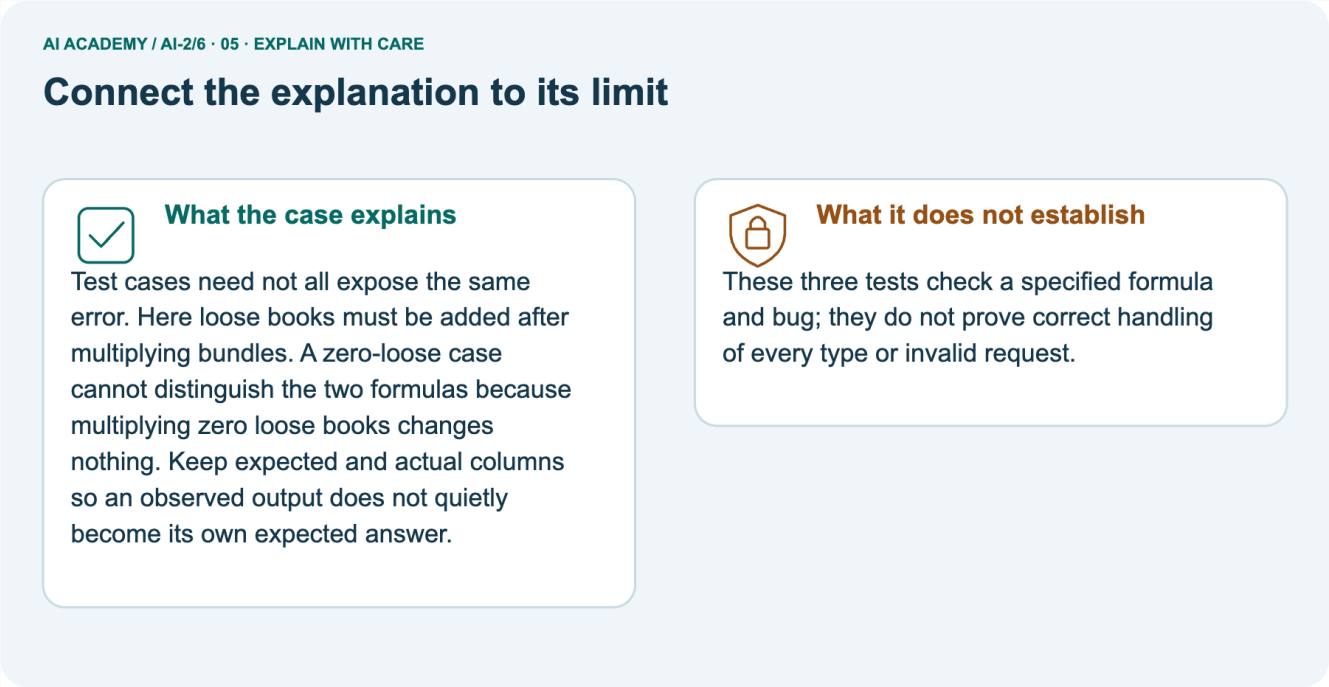
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: For (2,0), both return 12; a pass does not locate every bug. Changed condition: Repair the body to bundle_count*6+loose. Expected result: Expected results for (2,1),(0,3),(2,0) are 13,3,12.

Reason: Only bundles are groups of six; loose books are counted individually.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

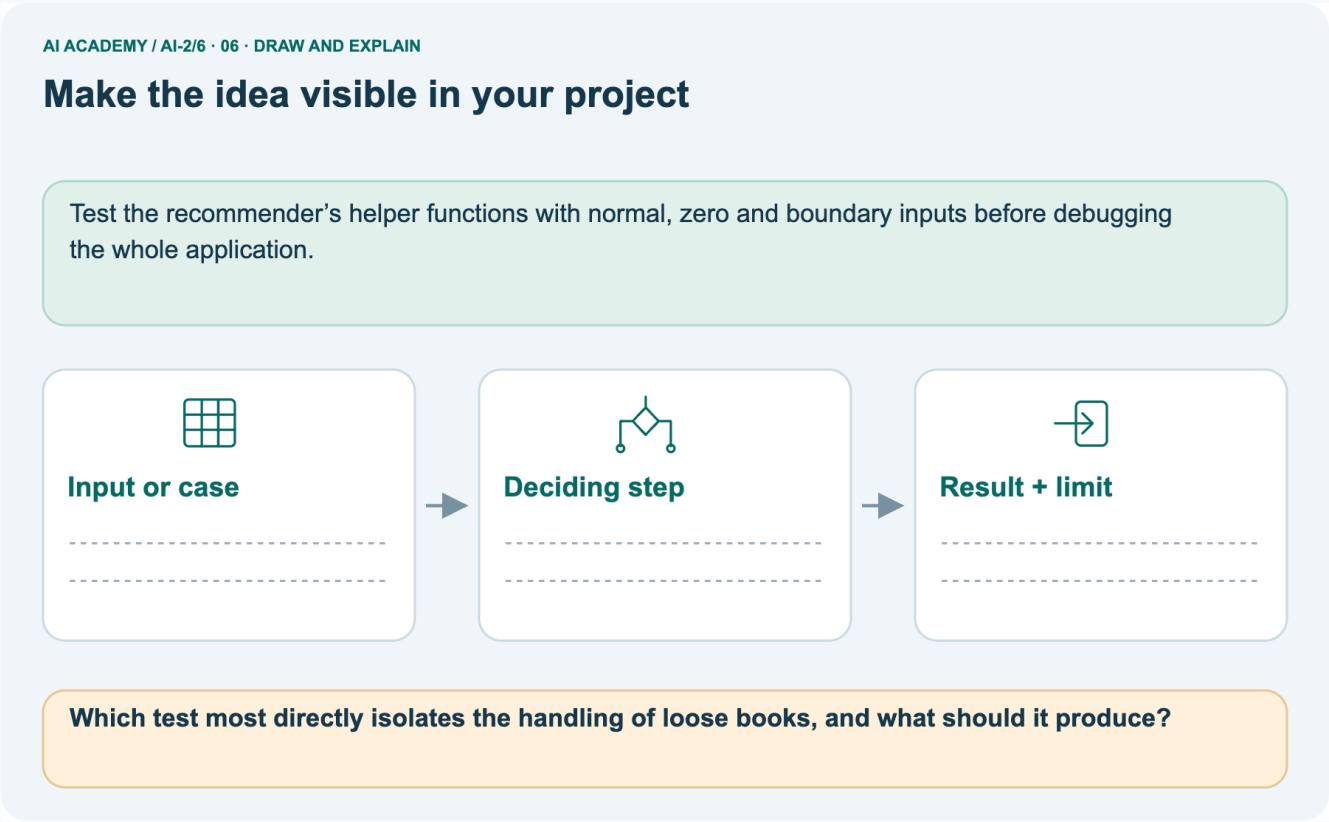
Test cases need not all expose the same error. Here loose books must be added after multiplying bundles. A zero-loose case cannot distinguish the two formulas because multiplying zero loose books changes nothing.

Keep expected and actual columns so an observed **output** does not quietly become its own expected answer.

Limit: These three tests check a specified formula and bug; they do not prove correct handling of every type or invalid request.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

Test the recommender’s helper functions with normal, zero and **boundary input**s before **debug**ging the whole application. Draw your **input**, deciding step, result and limit.

Supported question: Which test most directly isolates the handling of loose books, and what should it produce?

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Debugging Strategies: the case

02. Follow the deciding steps

03. Expected outputs versus a specific bug

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.